Master's thesis

Examining Floquet Quantum Error Correction Codes

Henri Wegener

supervised by Jens Eisert and Alex Townsend-Teague

December 2025
Freie Universität Berlin

Abstract

On most quantum hardware platforms, stabiliser codes with high measurement weights are difficult to implement fault-tolerantly. To overcome this we employ *Floquet codes*, a recently proposed class of dynamical QECC that extends stabiliser and subsystem codes by allowing their logical Pauli operators to evolve in time. In this thesis, we synthesise low-weight Floquet versions of the colour code with measurement weights of only one or two that can be laid out on a square lattice. We import the ZX calculus, a graphical language for representing and rewriting quantum circuits, to investigate two distinct Floquetification procedures: a naive rewrite strategy and a *fault-equivalent* rewrite scheme.
We prove that the latter preserves the code distance and the circuit's noise behaviour during Floquetification. For the first time, we perform threshold simulations on fault-equivalently Floquet codes and show that those rewrites alone do not by themselves yield strong performance under circuit-level noise.

Contents

1 Introduction

Fault-tolerant quantum error correction is essential for scalable quantum computing [1–7]. Several promising quantum error correction (QEC) protocols give a good trade-off among code parameters like encoding rate, distance and thresholds. The most widely used static error correction codes include CSS codes [8] like surface codes [9–11] and colour codes [4, 12, 13]. Recent experiments on surface-code architectures have achieved sub-threshold logical operations on superconducting qubits [14] and first logical gate primitives such as lattice surgery [15, 16] and code switching [17–19].

However, unwanted entanglement with the environment and measurement noise still dominate circuit-level performance, making the construction of more efficient quantum error correction codes (QECCs) a vital task [20]. A key hardware constraint is that high-weight stabiliser measurements are difficult to implement on most platforms and are therefore unfavourable. For indirect measurements, lower measurement weights mean that the number of entanglement operations with ancillary qubits is decreased, making the error-readout process less susceptible to error propagation and decoherence. For instance in transmon architecture on superconducting chips which directly measure the parity of qubits, measurements with a weight of more than two are significantly more difficult to implement [21].

A recently discovered remedy are *Floquet codes* [22, 23]; a class of dynamic error correction codes that generalise stabiliser codes and allow for non-commuting measurements [22]. Their logical qubits are generated dynamically over time and the codespace can be described at each time step by an instantaneous stabiliser group. Experiments show that Floquet codes can outperform their static counterparts [24] and enable fault-tolerant logical Clifford gates [25].

Floquet codes can be generated from existing stabiliser codes using the *ZX calculus* [26, 27], a graphical language for describing any linear map over a number of qubits [27, 28]. The formalism has proven useful in different research areas like measurement-based quantum computing [29–31], circuit synthesis and optimisation [32–34] and quantum error correction [28, 35, 36]. The ZX calculus has already been used to turn static CSS codes into dynamic ones [37, 38]. This is achieved by translating quantum circuits into ZX diagrams which come with a set of handy graphical rewriting rules allowing us to transform and simplify circuits. Recently, ZX rewrite rules and their properties have been at the center of intense research [37, 39–41] aiming to preserve code properties during the circuit manipulation. Rodatz et al. discovered a set of *fault-equivalent* rewrites [42] that preserve the code distance and the circuit's behaviour under noise [38].

In this thesis we use two distinct sets of rewrite rules to synthesise Floquet colour codes. To analyze the performance of fault-equivalent Floquet codes, for the first time, we perform circuit-level noise simulations using Stim [43], an efficient stabiliser circuit simulator. We find that fault-equivalent Floquet codes yield lower logical error rates than naively synthesised Floquet codes, but neither exhibits threshold behaviour in our simulations. Hence, we conclude that fault-equivalent rewrites alone do not improve the colour code's resilience to noise.

This thesis proceeds as follows: After revisiting QEC fundamentals and the

static stabiliser formalism [44], we introduce *ISG codes* of which stabiliser and Floquet codes are subclasses. We show how ISG codes can be described by a varying instantaneous stabiliser and logical group and how measurements affect those.
In the following we revisit the functionality of quantum decoders and explain the two efficient decoding algorithms used for our simulations.
Next, we introduce ZX diagrams and import the recently proposed *Pauli webs* [45], which we use to track code stabilisers and the propagation of errors. We introduce two different restricted rewrite rule sets and use them to *Floquetify* the 2D colour code, compile the resulting schedules, and run circuit-level simulations under realistic noise. After presenting our simulation results, we discuss possible sources of error that could influence the observed behaviour. Finally we look ahead to open questions and potential improvements.

2 Quantum Error Correction

Compared to classical computers, quantum hardware platforms are more susceptible to errors due to thermal or electromagnetic noise. In order to protect the quantum analogon to the classical bit, the *qubit*, from errors, it is encoded using redundancy. A qubit can be any quantum two-level system whose pure states can be represented by normalized vectors in a two-dimensional Hilbert space. Unlike duplicating the information of a classical bit on multiple instances, a qubit state $|\psi\rangle$ cannot simply be copied onto multiple qubits due to the *no-cloning theorem*[1]. This means there exists no unitary operation U_{clone} such that:

$$U_{\text{clone}}(|\psi\rangle \otimes |0\rangle) = |\psi\rangle \otimes |\psi\rangle \qquad \forall\, |\psi\rangle \tag{1}$$

Instead, we expand the Hilbert space of the qubit state and distribute its quantum information onto multiple *entangled* qubits, e.g.

$$\alpha\,|0\rangle + \beta\,|1\rangle \longrightarrow \alpha\,|000\rangle + \beta\,|111\rangle \tag{2}$$

In this way, it is possible to encode logical information in the physical hardware.

2.1 Basic Building Blocks

In quantum error correction, four Hilbert spaces are used to describe the logical data that is stored on the physical hardware and to protect it from being corrupted by external noise:

(s-i) The *physical space* $\mathcal{H}_{\text{phys}} = (\mathbb{C}^2)^{\otimes n}$ describes all possible physical states (corrupted or uncorrupted) of the n-qubit hardware.

(s-ii) The *codespace* is a subspace of the physical space $\mathcal{H}_{\text{code}} \subset \mathcal{H}_{\text{phys}}$. It is the space of states that will be used and projected during the computation.

(s-iii) The *logical space* $\mathcal{H}_{\text{log}} \cong (\mathbb{C}^2)^{\otimes k}$ is the abstract Hilbert space of the encoded quantum information. It is not tied to the physical qubit hardware, but instead stores the k *logical qubits* of the code. Equivalently to the elements of $\mathcal{H}_{\text{phys}}$, the elements of the logical space are referred to as *logical states* $|\psi\rangle_L$.

(s-iv) The *error spaces* $\mathcal{H}_i := e_i(\mathcal{H}_{\text{code}}) \subset \mathcal{H}_{\text{phys}}$ are the spaces of states one gets after applying a Pauli error e_i to an element of the codespace and thereby leaving it.

Quantum error correction is concerned with the maps relating these four spaces to one another:

[1] Assume there exists a map U_{clone} that copies arbitrary quantum states: $U_{\text{clone}}(|\psi\rangle \otimes |0\rangle) = |\psi\rangle \otimes |\psi\rangle$. Comparing the inner product of two pure states $|\psi\rangle$ and $|\phi\rangle$ before cloning $(\langle\psi| \otimes \langle 0|)(|\phi\rangle \otimes |0\rangle) = \langle\psi|\phi\rangle$ and after cloning $(\langle\psi| \otimes \langle\psi|)(|\phi\rangle \otimes |\phi\rangle) = (\langle\psi|\phi\rangle)^2$ implies that $\langle\psi|\phi\rangle \in \{0, 1\}$, so the states must be identical or orthogonal, violating the initial assumptions.

(m-i) The *encoding map* $\mathcal{N} : \mathcal{B}(\mathcal{H}_{\log}) \to \mathcal{B}(\mathcal{H}_{\text{code}})$ [2] unitarily maps density matrices ρ_L from the logical space into the codespace. (Describing quantum states as density matrices ρ generalises the pure state formalism, a thorough explanation can be found in Appendix A.1.)
In order to obtain information about the physical state of the system without performing projective measurements on a qubit subset called *data qubits*, one uses m *ancillary qubits* which can be directly measured, because they do not contain any logical information. Those are usually initialised and reset to the ground state $|0\rangle$ and are part of the encoding map of the code:

$$\mathcal{N} : \rho_L \mapsto \mathcal{N}(\rho_L \otimes (|0\rangle \langle 0|)^{\otimes m})\mathcal{N}^\dagger$$

After encoding the logical qubits into the physical realisation, the desired quantum process is performed before the quantum code is *decoded* by the inverse *decoding map* $\mathcal{D} = \mathcal{N}^\dagger$. Encoding and then decoding a logical state $|\psi\rangle_L \in \mathcal{H}_{\log}$ in the absence of uncorrectable noise acts as the identity: $\mathcal{D}(\mathcal{N}(|\psi\rangle_L)) = |\psi\rangle_L$.

(m-ii) Operations O that preserve the codespace $O : \mathcal{H}_{\text{code}} \to \mathcal{H}_{\text{code}}$ can be separated into two categories. The set of *stabilisers* $\mathcal{S}$ consists of Pauli operators s that act trivially on the codespace: $\mathcal{S} : s\,|\psi\rangle = |\psi\rangle \quad \forall\, |\psi\rangle \in \mathcal{H}_{\text{code}}, s \in \mathcal{S}$.

The set $\mathcal{L}$ of operators that act *non*-trivially on the codespace are called *logical operators*. These operators have correspondences in the logical space, where one wants to perform a computation on the logical data.

(m-iii) *Errors* e_i are operators that do not preserve the codespace: $e_i : \mathcal{H}_{\text{code}} \to \mathcal{H}_i \subset \mathcal{H}_{\text{phys}}$ They map elements of the codespace to a specific error space with index i. In order to theoretically be able to correct all errors, we assume them to be unitary (hence reversible) maps between the codespace and the respective error space. Note that this does not imply information on the actual physical noise the hardware is subject to.

(m-iv) The errors that can physically happen on qubit level are described by a *noise channel* $\mathcal{E}$. The operation $\mathcal{E} : \mathcal{B}(\mathcal{H}_{\text{phys}}) \to \mathcal{B}(\mathcal{H}_{\text{phys}})$ is a completely positive trace-preserving (CPTP) map modelling a specific noise source that is dependent on the hardware implementation of the quantum computer. It can be written in Kraus form as

$$\mathcal{E}(\rho) = \sum_r \tilde{e}_r \rho \tilde{e}_r^\dagger,$$

[2] For a finite-dimensional Hilbert space $\mathcal{H}$ the set of all bounded linear operators on that space is denoted by $\mathcal{B}(\mathcal{H})$. Specifically if $\dim(\mathcal{H}) = \mathrm{D}$, the set of all linear operators on that space is isomorphic to the set of all $D \times D$ complex matrices: $\mathcal{B}(\mathcal{H}) \cong \mathbb{C}^{D \times D}$.

where the Kraus operators $\tilde{e}_r$ can be chosen from (or expanded in) a basis error set $\{e_i\}$ but need not be unitary.

(m-v) The aim of quantum error correction is to detect and correct errors to prevent the logical information from being corrupted. This is done by performing suitable collective measurements, *syndrome measurements* that reveal whether the system is still in the codespace or has been shifted into a distinct error subspace. According to the result of the syndrome measurements, we perform a recovery operation to restore the system state. We can combine the syndrome measurement and the recovery operation into a single *error correction operation* $\mathcal{O} : \mathcal{B}(\mathcal{H}_{\text{phys}}) \to \mathcal{B}(\mathcal{H}_{\text{code}})$, mapping a corrupted state $\mathcal{N}(\rho_{\text{code}})$ back to the codespace:

$$\mathcal{O}(\mathcal{N}(\rho_{\text{code}})) = \rho_{\text{code}}$$

We say the error correction fails when $\mathcal{O}$ ends up applying a non-trivial logical operation to the encoded state. Such *logical errors* result from an incorrect syndrome-to-error assignment.

The four relevant Hilbert spaces and the mappings between them are visualized in Fig. 1.

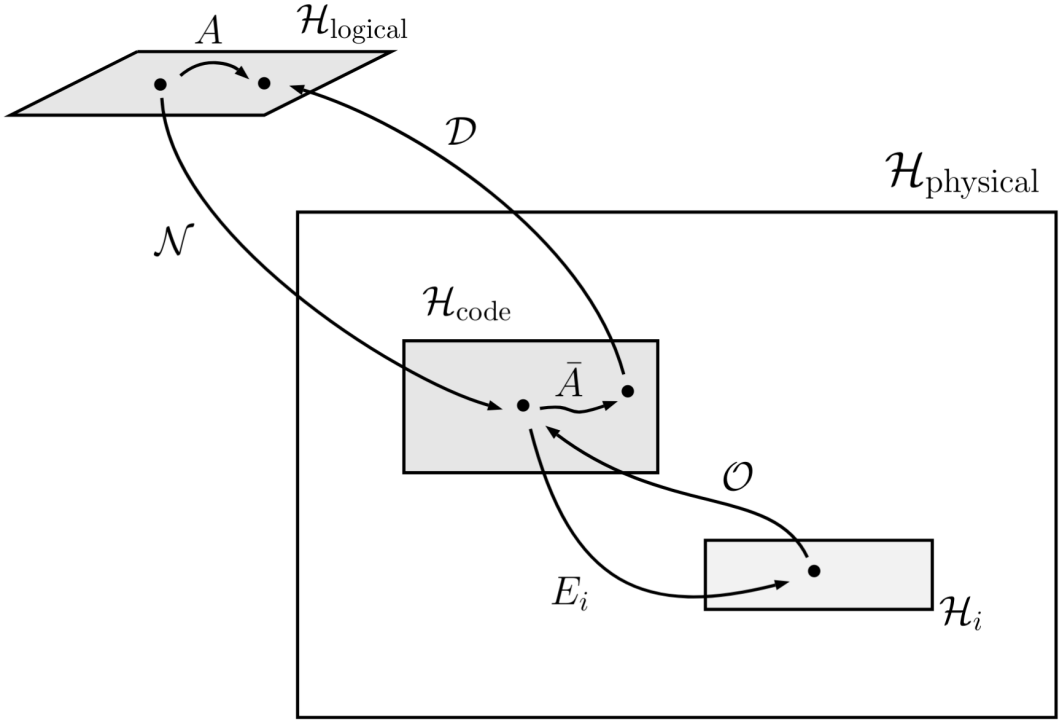


Figure 1: The relevant Hilbert spaces to describe a quantum computer as introduced in **(s-i)-(s-iv)** and the maps **(m-i)-(m-v)** representing their relations to each other. An abstract logical operation $A \in \mathcal{B}(\mathcal{H}_{\text{log}})$ changing one logical state into another has a physical representation $\bar{A} \in \mathcal{B}(\mathcal{H}_{\text{code}})$ in the codespace, revealing the actual gates performed on qubit level supporting that gate.

Quantum error correction codes (QECC) are characterized by their code parameters

$$[\![n, k, d]\!] \tag{3}$$

Here, n is the qubit number, $k = \log_2(\dim(\mathcal{H}_{\text{code}}))$ is the number of encoded logical qubits and the *distance* d is the minimum weight[3] of any Pauli operator that preserves the codespace but acts non-trivially on the logical information.

Different hardware platforms are subject to different kinds of physical noise. In order to adequately simulate a QECC's performance, we have to consider how to computationally imitate this physical noise. In Appendix A.3 we describe more precisely what these *error channels* look like and what physical noise phenomena they describe. Then we explain how we incorporate the most realistic noise model called *circuit-level noise* in our simulations in Appendix A.4.

2.2 Static Stabiliser Formalism

As introduced in section Section 2.1, a QEC code with k logical qubits is defined in a 2^k-dimensional subspace $\mathcal{H}_{\text{code}}$ of the 2^n-dimensional physical Hilbert space $\mathcal{H}_{\text{phys}}$ in which the encoded states of the code are stored. This codespace $\mathcal{H}_{\text{code}}$ can equivalently be defined as the $(+1)$ eigenspace of the operators in the stabiliser group $\mathcal{S}$. $\mathcal{S}$ is an abelian subgroup of rank $r = n - k$ of the n-fold Pauli group $\mathcal{P}^{\otimes n}$ [4].

The operators $s \in \mathcal{S}$ are said to *stabilize* the states $|\psi\rangle$ in the codespace if they fulfil the following equation:

$$s\,|\psi\rangle = (+1)\,|\psi\rangle \tag{4}$$

In order for the codespace to be non-trivial, we have the condition that $-\mathbb{I} \notin \mathcal{S}$. As a result, $\mathcal{S}$ is abelian and any two stabilisers s_i, s_j in the group commute[5]:

$$[s_i, s_j] = 0 \tag{5}$$

QEC codes that can be defined by a stabiliser group are called *stabiliser codes*

Example 2.1 An intuitive example is given by the $[\![4, 2, 2]\!]$ code, which is the smallest stabiliser code that detects single qubit errors. Its codespace is spanned by the codewords $|\overline{\psi}\rangle$ given by

$$\mathcal{H}_{code[\![4,2,2]\!]} = \text{span}\left\{\begin{array}{l} |\overline{00}\rangle = \frac{1}{\sqrt{2}}\left(|0000\rangle + |1111\rangle\right) \\ |\overline{01}\rangle = \frac{1}{\sqrt{2}}\left(|0110\rangle + |1001\rangle\right) \\ |\overline{10}\rangle = \frac{1}{\sqrt{2}}\left(|1010\rangle + |0101\rangle\right) \\ |\overline{11}\rangle = \frac{1}{\sqrt{2}}\left(|1100\rangle + |0011\rangle\right) \end{array}\right\} \tag{6}$$

[3] The weight of a Pauli operator is the number of qubits on which it acts nontrivially. For a Pauli string the weight is the count of operators that are not the identity.

[4] The n-fold Pauli group is given by $\mathcal{P}^{\otimes n} = \langle i, X, Z\rangle^{\otimes n}$

[5] The commutator of two linear operators A and B in a complex Hilbert space $\mathcal{H}$ is defined as the following operator identity: $[A, B] = AB - BA$

One can easily verify that these states are stabilized by the operators generated by

$$\mathcal{S}_{[\![4,2,2]\!]} = \langle X_1X_2X_3X_4,\ Z_1Z_2Z_3Z_4\rangle \tag{7}$$

As defined in **(m-ii)**, the logical operator group $\mathcal{L}$ consists of equivalence classes of Pauli operators that commute with every stabiliser, where two such operators are considered the same logical operator if they differ by a stabiliser.
A possible choice of logical operators for the $[\![4,2,2]\!]$ code is given by the following generators:

$$\begin{aligned}\mathcal{L}_{[\![4,2,2]\!]} = \langle \bar{i},\ \bar{x}_1 &= X_1X_3,\\ \bar{z}_1 &= Z_1Z_4,\\ \bar{x}_2 &= X_2X_3,\\ \bar{z}_2 &= Z_2Z_4\rangle\end{aligned} \tag{8}$$

Here $\bar{i}$ denotes the coset of the complex number; it acts trivially on the code. These operators act as maps between the codewords $|\overline{\psi}\rangle$, e.g.

$$\bar{x}_1 |\overline{00}\rangle = X_1X_3 \cdot (\frac{1}{\sqrt{2}}(|0000\rangle + |1111\rangle)) \tag{9}$$

$$= \frac{1}{\sqrt{2}}(|1010\rangle + |0101\rangle) \tag{10}$$

$$= |\overline{10}\rangle \tag{11}$$

As stated above, it is possible to detect any single qubit error on any of the four qubits by measuring the stabiliser generators. For instance, suppose a bitflip (X-)error occurred on the first qubit on the encoded state $|\overline{\psi}\rangle$. We can derive the measurement outcomes as follows:

$$X_1X_2X_3X_4(X_1 |\overline{\psi}\rangle) = X_1(X_1X_2X_3X_4 |\overline{\psi}\rangle) = X_1 |\overline{\psi}\rangle \tag{12}$$

$$Z_1Z_2Z_3Z_4(X_1 |\overline{\psi}\rangle) = -X_1(Z_1Z_2Z_3Z_4 |\overline{\psi}\rangle) = -X_1 |\overline{\psi}\rangle \tag{13}$$

Since $-Z_1Z_2Z_3Z_4$ is a stabiliser of $X_1 |\overline{\psi}\rangle$, measuring this operator on the corrupted state $X_1 |\overline{\psi}\rangle$ will deterministically give outcome -1. In fact any odd-weight Pauli error will cause a (-1) measurement outcome in at least one of the stabiliser measurements.

2.3 Dynamic Stabiliser Formalism/ISG codes

Now that we have established a link between group theory and the underlying linear algebra, we can stick with a group-theoretic description. A system of n qubits that is described by a stabiliser group $\mathcal{S}$ with rank r s.t. $-\mathbb{1} \notin \mathcal{S}$ can be described by a group $\mathcal{G}$ that is isomorphic to $\mathcal{P}^{\otimes(n-r)}$. One such group is given by $\mathcal{G} = N(\mathcal{S})/\mathcal{S} \cong \mathcal{P}^{\otimes(n-r)}$, which is the logical Pauli group of the code. For a stabiliser group $\mathcal{S}$ the logical Pauli group is the quotient group which consists of left cosets in the normalizer $N(\mathcal{S})$ of $\mathcal{S}$ in $\mathcal{P}^{\otimes n}$. The *normalizer group* of $\mathcal{S}$ is defined as

$$N(\mathcal{S}) = \{p \in \mathcal{P}^{\otimes n} : p\mathcal{S}p^{-1} = \mathcal{S}\} \tag{14}$$

and consists of the n-qubit operators that commute with all stabilisers in $\mathcal{S}$ [6]. Its elements are the Pauli operators that are multiplied from the left to all stabiliser generators: $p\mathcal{S} = \{ps | s \in \mathcal{S}\}$ and will be denoted $\bar{p}$ in the following. It is possible to represent the Pauli group on n qubits as $\mathcal{P}^{\otimes n} = \langle \mathbb{1}, X_1, Z_1, X_2, Z_2, ..., X_n, Z_n \rangle$. Note that operators with the same index anticommute while operators on different qubits always commute. This property can be transferred to any group isomorphic to $\mathcal{P}^{\otimes n}$. For instance, it is possible to represent the logical Pauli group $N(\mathcal{S})/\mathcal{S}$ as $\langle \iota, x_1, z_1, x_2, z_2, ..., x_n, z_n \rangle$ with the same commutativity relations.
Having clarified the foundation, we can now define *instantaneous stabiliser group codes*.

Definition 2.1. *An **instantaneous stabiliser group (ISG) code** on n physical qubits is defined by an ordered measurement schedule $\mathcal{M} = [\mathcal{M}_0, \mathcal{M}_1, \mathcal{M}_2, \dots]$, where each $\mathcal{M}_j \subseteq \mathcal{P}^{\otimes n}$ is an Abelian subgroup of the n-qubit Pauli group $\mathcal{P}^{\otimes n}$. The schedule $\mathcal{M}$ may be of either finite or infinite length. If it is finite with length ℓ, then the indices j in $\mathcal{M}_j$ are taken modulo ℓ, making the schedule periodic.*

For each timestep $t \in \mathbb{Z}$, we define a subgroup $\mathcal{S}_t \subseteq \mathcal{P}^{\otimes n}$, called the *instantaneous stabiliser group* at time t, recursively as follows:

- For all $t < 0$, $\mathcal{S}_t = \{\mathbb{1}\}$ is the trivial group.
- For $t \geq 0$, one forms $\mathcal{S}_t$ from $\mathcal{S}_{t-1}$ by measuring a generating set of $\mathcal{M}_t$. The effect of such measurements is determined using the stabiliser formalism and does not depend on the particular generating set chosen for $\mathcal{M}_t$. The procedure of measuring in the stabiliser formalism is explained in Section 2.4.
- There will always exist a $T \in \mathbb{Z}_{\geq 0}$ such that for $t \geq T$ the group $\mathcal{S}_t$ has a fixed rank r and the logical Pauli group has fixed rank k. We call the code *established*.

Once the code is established at $t = T$, it encodes $k = n - r$ logical qubits. The *instantaneous distance* d of the code is given by the minimum weight of any element in the logical Pauli group $N(\mathcal{S})/\mathcal{S}$ at $t \geq T$. This serves as an upper bound for the actual code distance, which can take lower values because an undetectable logical error of lower weight might spread over multiple time steps.

[6] The group of all elements that commute with $\mathcal{S}$ is actually the *centralizer* of $\mathcal{S}$. However for a stabiliser group $\mathcal{S}$, the normalizer group coincides with the centralizer group. Since the normalizer group is defined as the group of elements that conjugate $\mathcal{S}$ to itself in Eq. 14 and conjugation acts as commutation in the Pauli group, both groups describe the same elements.

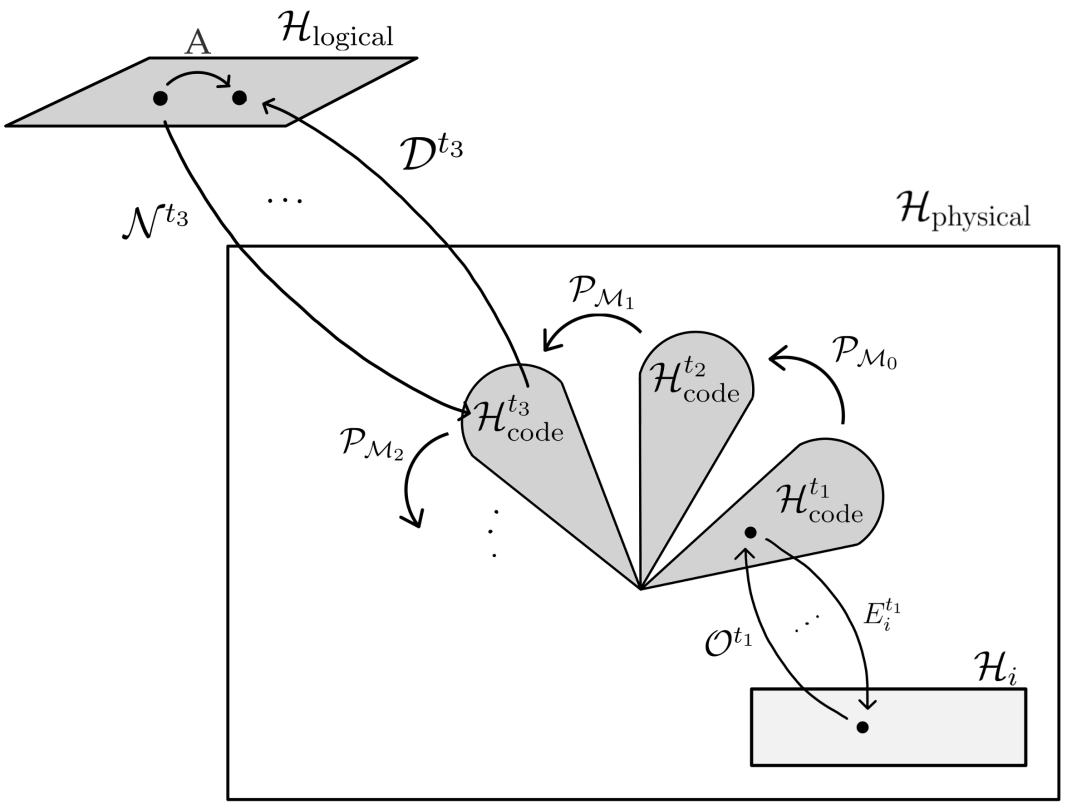


Figure 2: The relevant Hilbert spaces for ISG codes, equivalent to Fig. 1. The instantaneous stabiliser group $\mathcal{S}_t$ at timestep t stabilizes the instantaneous codespace $\mathcal{H}^t_{\text{code}}$. The codespaces at different timesteps are mapped to one another by measurement-related maps $\mathcal{P}_{\mathcal{M}_i}$. Errors E_i (see **(m-iii)**) and error correction operations $\mathcal{O}$ (see **(m-v)**) are now dependent on the current Hilbert space at time t. Equally for the encoding map $\mathcal{N}$ and the decoding map $\mathcal{D}$ (see **(m-i)**). However the abstract logical Hilbert space $\mathcal{H}_{\text{logical}}$ remains constant. For Floquet codes (see Sec. 2.3.1), the mapping $\mathcal{P}_{\mathcal{M}_i}$ is periodic, so after a finite number of timesteps, we end up in the initial codespace again.

2.3.1 Floquet Codes

Floquet codes are a subclass of ISG codes with the characteristic that the measurement schedule $\mathcal{M}$ has a finite period. Stabiliser codes can be seen as a subclass of Floquet codes where the measurement schedule has a length of one. The first Floquet code discovered with dynamically generated logical qubits is the Honeycomb code introduced by M. Hastings et al.[22]. [...]

2.4 Measurements in the Stabiliser Formalism

In order to show how a code like the $[\![4, 2, 2]\!]$ code can be generated by repeatedly measuring Hermitian operators, it is important to recall how measurements work in the stabiliser formalism. While it is possible to track the changes of the stabiliser group $\mathcal{S}$ during a measurement protocol, it is analogously possible to track the changes of the logical Pauli group $N(\mathcal{S})/\mathcal{S}$. In the following, we will simultaneously describe the code evolution in terms of both $\mathcal{S}$ and $N(\mathcal{S})/\mathcal{S}$.

Before any measurement, we are given an initial stabiliser group $\mathcal{S} = \langle\rangle$ and the corresponding initial logical Pauli group

$$N(\mathcal{S})/\mathcal{S} = \langle \bar{i}, \bar{X}_1, \bar{Z}_1, \bar{X}_2, \bar{Z}_2, ..., \bar{X}_k, \bar{Z}_k \rangle$$

As stated above, the bars $\bar{p}$ indicate that the operators in $N(\mathcal{S})/\mathcal{S}$ are multiplied from the left to the stabiliser group $\mathcal{S}$. The logical operators appear in anti-commuting pairs $\{\bar{X}_i, \bar{Z}_i\}$, but operators with different indices always commute. The measurement of a hermitian operator P with outcome $m \in \{-1, 1\}$ generates a new post-measurement stabiliser group $\mathcal{S}'$ and logical Pauli group $N(\mathcal{S}')/\mathcal{S}'$. Depending on the commutativity between P and the elements of $\mathcal{S}$, we can differentiate three cases:

(c-i) P commutes with all elements in $\mathcal{S}$ but $\pm P \notin \mathcal{S}$. This means that the measurement outcome $m \in \{-1, 1\}$ is non-deterministic [7] and the stabiliser group is extended by one element: $\mathcal{S}' = \langle mP, s_1, ..., s_r \rangle$. Equivalently $\bar{P} \in N(\mathcal{S})/\mathcal{S}$ (and $\bar{P} \neq \pm\bar{\mathbb{1}}$). WLOG it is always possible to manipulate the logical Pauli group such that $\bar{P} = \bar{X}_1$. We can then omit the pair $\{\bar{X}_1, \bar{Z}_1\}$ from the group and have: $N(\mathcal{S}')/\mathcal{S}' = \langle \bar{i},\ \bar{X}_2,\ \bar{Z}_2,\ \bar{X}_3,\ \bar{Z}_3,\ ...,\ \bar{X}_k,\ \bar{Z}_k, \rangle$

(c-ii) P commutes with all elements in $\mathcal{S}$ *and* $\pm P$ is contained in $\mathcal{S}$. Equivalently, $\bar{P}$ is either $\bar{\mathbb{1}}$ or $-\bar{\mathbb{1}}$. As the code is already stabilized by this operator, the measurement outcome is deterministic and the stabiliser group as well as the logical group remain unchanged: $\mathcal{S}' = \mathcal{S}$; $N(\mathcal{S}')/\mathcal{S}' = N(\mathcal{S})/\mathcal{S}$.

(c-iii) P *anti*-commutes with at least one stabiliser. WLOG we can assume that P anti-commutes only with $s_1 \in \mathcal{S}$. Then the measurement outcome m is truly random and we replace s_1 by mP in the stabiliser group: $\mathcal{S}' = \langle mP, s_2, s_3, ..., s_r \rangle$. By definition of $N(\mathcal{S})/\mathcal{S}$, this means that $\bar{P}$ is not contained in the logical Pauli group. It is always possible to choose new coset representatives such that $\bar{P}$ commutes with all of them and we find that $N(\mathcal{S}')/\mathcal{S}' = \langle \bar{i}, \bar{X}_1, \bar{Z}_1, \bar{X}_2, \bar{Z}_2, ..., \bar{X}_k, \bar{Z}_k \rangle$ where $\bar{X}_j$ and $\bar{Z}_j$ denote $X_j\mathcal{S}'$ and $Z_j\mathcal{S}'$.

In case **(c-i)** we assumed a group theoretic property about the logical Pauli group which can be phrased as the following proposition:

Proposition 2.1. *After measuring a Pauli operator P that commutes with all elements in $\mathcal{S}$ but is not contained in $\mathcal{S}$: $\pm P \notin \mathcal{S}$, by definition of the logical group, it follows that $\bar{P} \in N(\mathcal{S})$. Then it is always possible to find new representatives of the logical group $N(\mathcal{S})/\mathcal{S} = \langle \bar{i},\ \bar{X}_2,\ \bar{Z}_2,\ \bar{X}_3,\ \bar{Z}_3,\ ...,\ \bar{X}_k,\ \bar{Z}_k, \rangle$ such that $\bar{X}_1 = \bar{P}$.*

Proof We are in case **(c-i)**. Let $N(\mathcal{S})/\mathcal{S}$ be the logical Pauli group of the stabiliser group $\mathcal{S}$ and $P \in \mathcal{P}^{\otimes n}$ be a Pauli operator with $[P, \mathcal{S}] = 0$. First we find an operator $\bar{Q} \in N(\mathcal{S})/\mathcal{S}$ which *anti*commutes with $\bar{P}$. This is always possible,

[7] Depending on the arbitrary pre-measurement state, the outcome can either be deterministic or random.

which can be shown by contradiction: If no $\bar{Q}$ existed such that $\{\bar{Q}, \bar{P}\} = 0$, then $\bar{P}$ commutes with every element in $N(\mathcal{S})/\mathcal{S}$ and thus acts trivially on the codespace, i.e. is a stabiliser. But we assumed $P \notin \mathcal{S}$.

We now choose $\bar{X}_1 = \bar{P}$ and $\bar{Z}_1 = \bar{Q}$ to be the elements from which we rebuild our logical Pauli group: We multiply $\bar{Q}$ with all $N \in N(\mathcal{S})/\mathcal{S}$ for which $\{\bar{N}, \bar{P}\} = 0$. This will restore the (anti-)commutation relations in the logical Pauli group because the product of two operators that anticommute with P will commute with P:

$$\begin{aligned}[QN, P] &= Q[N, P] - [Q, P]N \\ &= 2QNP + 2QPN \\ &= 2QNP - 2QNP \\ &= 0\end{aligned}$$

Where we used $\{\bar{N}, \bar{P}\} = 0$ in the penultimate step. The resulting group has the same structure as $N(\mathcal{S})/\mathcal{S}$ where each element anticommutes with one other element but commutes with every other element.

□

The case distinction for measurements in the stabiliser formalism is summarized in the following table 2.4.

case	measurement outcome m	stabiliser group update	logical Pauli group update
(i) $[P, \mathcal{S}] = 0$ and $\pm P \notin \mathcal{S}$	non-deterministic	$\mathcal{S}' = \langle mP, S_1, ..., S_r \rangle$	Assume WLOG that $\bar{P} = \bar{X}_1$ then $N(\mathcal{S}')/\mathcal{S}' = \langle \bar{i}, \bar{X}_2, \bar{Z}_2, ..., \bar{X}_k, \bar{Z}_k \rangle$
(ii) $[P, \mathcal{S}] = 0$ and $\pm P \in \mathcal{S}$	deterministic	$\mathcal{S}' = \mathcal{S}$	$\bar{P}$ is $\bar{\mathbb{1}}$ or $-\bar{\mathbb{1}}$ and $N(\mathcal{S}')/\mathcal{S}' = N(\mathcal{S})/\mathcal{S}$.
(iii) $\exists S_i \in \mathcal{S} : \{P, S_i\} = 0$	truly random	$\mathcal{S}' = \langle mP, S_2, ..., S_r \rangle$	$N(\mathcal{S}')/\mathcal{S}' = \langle \bar{i}, \bar{X}_1, \bar{Z}_1, \bar{X}_2, \bar{Z}_2, ..., \bar{X}_k, \bar{Z}_k \rangle$

2.4.1 Example: Dynamically building the $[\![4, 2, 2]\!]$ Code

As an example we can generate the $[\![4, 2, 2]\!]$ code by repeatedly measuring the code stabilisers and track the changes in the stabiliser and logical Pauli groups. The trivial initial stabiliser group is given by $\mathcal{S}_{-1} = \langle\rangle$ and the initial logical Pauli group has its maximal size:

$$\begin{aligned}N(\mathcal{S}_{-1})/\mathcal{S}_{-1} = \langle & iS_{-1}, X_1S_{-1}, Z_1S_{-1}, \\ & X_2S_{-1}, Z_2S_{-1}, \\ & X_3S_{-1}, Z_3S_{-1}, \\ & X_4S_{-1}, Z_4S_{-1} \rangle\end{aligned}$$

In the first step we measure the Pauli operator $P_0 = Z_1Z_2Z_3Z_4$. P_0 trivially commutes with $\mathcal{S}_{-1}$ and is not contained in it. Hence we are in case **(c-i)** and the measurement outcome a is not deterministic. As per the rules in table 2.4,

we have $\mathcal{S}_0 = \langle aZ_1Z_2Z_3Z_4 \rangle$ where $a \in \{-1, 1\}$ is the measurement outcome. We choose new coset representatives of $N(\mathcal{S}_{-1})/\mathcal{S}_{-1}$ such that $\bar{Z}_1 = P_0$ while maintaining the (anti-)commutation relations inside the group:

$$\begin{aligned} N(\mathcal{S}_{-1})/\mathcal{S}_{-1} = \langle & iS_{-1}, X_4S_{-1},\ Z_1Z_2Z_3Z_4S_{-1}, \\ & X_1X_4S_{-1},\ Z_1S_{-1}, \\ & X_2X_4S_{-1},\ Z_2S_{-1}, \\ & X_3X_4S_{-1},\ Z_3S_{-1} \rangle \end{aligned}$$

We can now remove $\bar{X}_1 = X_4S_{-1}$ and $\bar{Z}_1 = Z_1Z_2Z_3Z_4S_{-1}$ from the set and end up with

$$\begin{aligned} N(\mathcal{S}_0)/\mathcal{S}_0 = \langle & iS_0, X_1X_4S_0,\ Z_1S_0, \\ & X_2X_4S_0,\ Z_2S_0, \\ & X_3X_4S_0,\ Z_3S_0 \rangle \end{aligned}$$

In the second step, we measure the Pauli operator $P_1 = X_1X_2X_3X_4$. This operator commutes with the single element in $\mathcal{S}_0$ and is not contained in it. Again, we are in case **(c-i)** and the outcome $b \in \{-1, 1\}$ is non-deterministic. We can add the operator to our stabiliser group and obtain $\mathcal{S}_1 = \langle aZ_1Z_2Z_3Z_4,\ bX_1X_2X_3X_4 \rangle$. Again we find new coset representatives of $N(\mathcal{S}_0)/\mathcal{S}_0$ such that $\bar{X}_2 = P_1$ while obeying the (anti-)commutation relations:

$$\begin{aligned} N(\mathcal{S}_0)/\mathcal{S}_0 = \langle & iS_0, X_1X_2X_3X_4S_0,\ Z_1S_0, \\ & X_2X_4S_0,\ Z_1Z_2S_0, \\ & X_3X_4S_0,\ Z_1Z_3S_0 \rangle \end{aligned}$$

We can remove the pair $\bar{X}_2 = X_1X_2X_3X_4S_0$ and $\bar{Z}_2 = Z_1S_0$ from the set and end up with

$$\begin{aligned} N(\mathcal{S}_1)/\mathcal{S}_1 = \langle & iS_1, X_1X_4S_1,\ Z_1Z_3S_1, \\ & X_2X_4S_1,\ Z_2Z_3S_1 \rangle \end{aligned}$$

Any further stabiliser measurement of the operators $Z_1Z_2Z_3Z_4$ and $X_1X_2X_3X_4$ will yield deterministic outcomes as we are in case **(c-ii)**. If not perturbed by any errors, the stabiliser group and the logical Pauli group will remain constant and the code is established. This property of deterministic outcomes for specific measurements for $t > T$ allows us to detect errors.

2.5 Quantum Decoders

After describing how noise acts on a quantum code and how it can be modelled by an error channel $\mathcal{E}$, the question arises how we can detect and correct such errors. As introduced in the previous section, measuring stabilisers gives us information about the quantum state of the qubit without perturbing it. Those stabiliser measurements produce a syndrome $\mathbf{s}$ that flags whether the quantum state left the codespace.

A *decoder* is a classical algorithm that takes as input the syndrome data and returns a recovery operation for the error. The performance of the decoder depends on how successfully the proposed recovery operation actually restores the state to the codespace.

In this section we will revisit the basic functionality of a decoding algorithm before explaining in detail the two decoders used in our simulations, namely *Chromobius* and *belief propagation* (BP).

2.5.1 Fundamentals of Quantum Decoding

Given a stabiliser code with stabiliser group $\mathcal{S}$ acting on n qubits, an error channel $\mathcal{E}$ induces a distribution over Pauli errors e_i (henceforth only referred to as e). The error operations map a state of the codespace to an error subspace:

$$e_i : \mathcal{H}_{\text{code}} \to \mathcal{H}_i \tag{15}$$

After performing a round of r stabiliser measurements, one can extract a syndrome in the form of a classical bit string:

$$\vec{s} = (s_1, ..., s_r) \in \{\pm 1\}^r \tag{16}$$

where each s_i is the eigenvalue measured of stabiliser generator $s_i \in \mathcal{S}$. The aim of any decoding algorithm can be phrased as Bayesian inference: Find a recovery operator r such that the probability that the combined operator, the *residual*, $r \cdot e$ acts trivially on the codespace (i.e. is a stabiliser) is maximised:

$$r^\star = \arg\max_r \sum_{\substack{e:\, s(e)=s \\ re \in \mathcal{S}}} P(e). \tag{17}$$

where $P(e)$ is the prior probability that e occurred and $s : \mathcal{P}^{\otimes n} \to \mathbb{F}_2^r$ is the map relating the Pauli errors to their respective syndromes. This means that, among all physical error patterns that could have produced the syndrome s, the decoder chooses the most likely one. This approach is called *maximum-likelihood* (ML) decoding. The decoder fails if the residual acts non-trivially on the codespace, i.e. is a logical operator. In this case the error causes the logical information stored in the codespace to be altered unnoticed.

Unfortunately, optimal ML decoding is computationally hard (even NP-hard [46]). Therefore there are several approaches that use approximations or other workarounds to make decoding tractable.

2.5.2 Graph-based Decoding

In graph-based decoding, the decoder does not operate directly on stabiliser operators but on *detection events* derived from *detectors*, which will be introduced more thoroughly in Ch. 3.3.1. Detectors are sets of measurements whose multiplied outcomes are deterministic in the absence of errors. If the detector evaluates to -1, we say that a *detection event* has occurred. Usually it is possible to find a

sufficiently large set of overlapping detectors on a code such that error locations at any point in space-time are covered by at least one detector.

The *detector error model* (DEM) is a map between elementary error processes and the detectors they flip. Figure 3 shows the detector model of a Floquetified $[\![4, 2, 2]\!]$ code. Individual errors are composed to get a probability for a specific detector flip. If the flip of two detectors is *correlated*, that is indicated by a hat ^.

```
error(0.02741843737473917755) D21 ^ D23 L0 ^ D20 L0
error(0.02741843737473917755) D21 ^ D23 ^ D20
error(0.1541807407407407571) D22
error(0.1541807407407407571) D22 L0
error(0.2918338533456616424) D23
error(0.2918338533456616424) D23 L0
error(0.02741843737473917755) D23 L0 ^ D13
error(0.02741843737473917755) D23 L0 ^ D20 L0
error(0.1228589094000363119) D23 L0 ^ D21
error(0.02741843737473917755) D23 ^ D14
error(0.02741843737473917755) D23 ^ D20
error(0.1228589094000363119) D23 ^ D21
```

Figure 3: Excerpt from the DEM of a Floquetified $[\![4, 2, 2]\!]$ code. Individual error likelihoods are combined to give the probability of a specific detector violation.

Graphically, the DEM can be interpreted as a bipartite graph connecting errors to detectors.

For most topological QECCs, due to the locality of stabilisers, Pauli errors cause detection events in small clusters in the graph which is why *minimum-weight-perfect-matching* (MWPM) algorithms can be applied to find the minimum-cost path between these events. The resultant matching of detection events is then mapped back to the underlying physical errors and a recovery operation is returned.

This picture underlies many prominent decoding approaches such as Möbius-type decoders. An explanation and a discussion of the decoders implemented for this thesis are given in Appendix A.2.

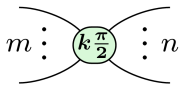
3 ZX Calculus

The *ZX calculus* is a graphical language for representing and rewriting quantum circuits. It was first introduced by B. Coecke et al. [27] and has been applied in a wide range of scientific fields ever since [28–36]. It has been proven recently that any equality between pure qubit linear maps can be derived graphically with the ZX calculus by adding a single additional axiom [47].

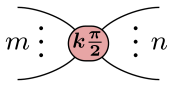
3.1 Preliminaries

In this section we briefly explain the basic generators and fundamental rewrite rules of the ZX calculus. To visualise quantum computations as string diagrams we introduce *ZX diagrams* which consist of a number of building blocks connected by wires. ZX diagrams come with a powerful set of rewrite rules enabling us to prove any true equality between linear maps on qubits.

The core components of this framework are green *Z-spiders* and red *X-spiders*, from which the name "ZX" originates. They are defined as follows:

Z spider: 

$$:= \quad |0\rangle^{\otimes n}\langle 0|^{\otimes m} + e^{ik\frac{\pi}{2}}|1\rangle^{\otimes n}\langle 1|^{\otimes m} \tag{18}$$

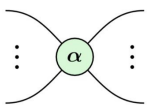
X spider: 

$$:= \quad |+\rangle^{\otimes n}\langle +|^{\otimes m} + e^{ik\frac{\pi}{2}}|-\rangle^{\otimes n}\langle -|^{\otimes m} \tag{19}$$

where the *spider phase* is determined by $k \in \mathbb{R}$. Spiders can have any number of input and output legs, which correspond to qubit ports.

Definition 3.1. ***Clifford ZX diagrams*** *are ZX diagrams in which the phases of all spiders are restricted to be multiples of* $\frac{\pi}{2}$.

In this work, we will almost exclusively work with Clifford ZX diagrams. Spiders are nothing but visual representations of a $2^n \times 2^m$ matrix in the respective X or Z basis.



$$= \begin{pmatrix} 1 & 0 & & \\ 0 & 0 & \ddots & \\ & \ddots & 0 & 0 \\ & & 0 & e^{i\alpha} \end{pmatrix}$$

Figure 4: Matrix representation of a Z spider.

Another important building block is the *Hadamard gate*

$$H = |0\rangle\langle +| + |1\rangle\langle -| \tag{20}$$

which is represented as a yellow box  in the ZX calculus. Additionally we introduce the (non-Clifford) *generalized Hadamard gate* which has an arbitrary number of inputs and outputs:

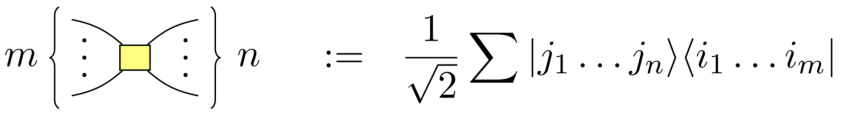


Figure 5: A generalized Hadamard gate with m inputs and n outputs as a ZX diagram and in Dirac notation. ZX diagrams containing this building block with $m, n \in \mathbb{N}$ s.t. $(m, n) \neq (1, 1)$ are non-Clifford (see Def. 3.1).

The generalized Hadamard gate represents a matrix where all entries are equal to 1, except the bottom right element, which is -1.
With those building blocks, it is possible to represent a large number of quantum maps as ZX diagrams. Some examples, e.g. the CNOT gate, the phase gate or Pauli measurements, are shown in Fig. 6.

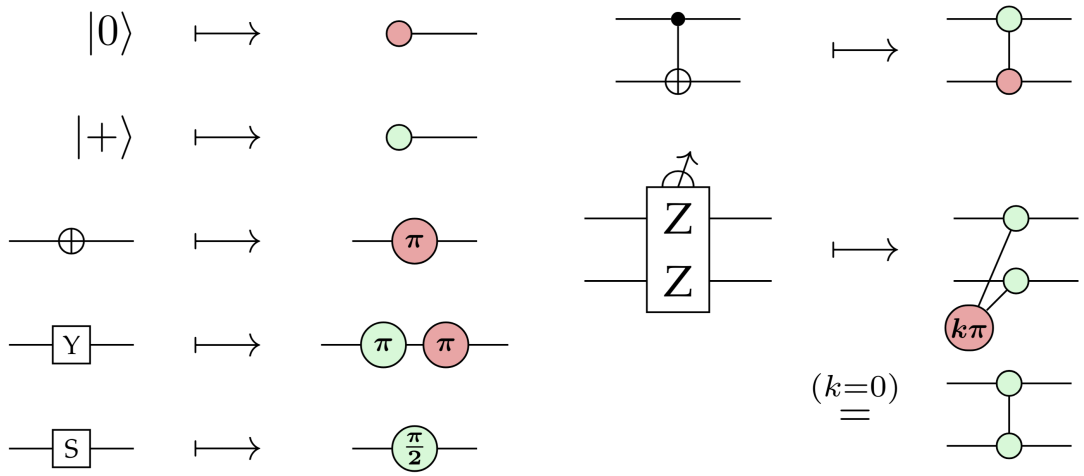


Figure 6: From top left to bottom right: A single red spider with one outgoing leg represents the case of $m = 0, n = 1$ in Eq. 18 and Eq. 19. Depending on the basis (colour), this represents the single-qubit $|0\rangle$ (for X-spider) or $|+\rangle$ (for Z-spider) state. The *NOT* gate is represented by a single X-spider with phase π. Via Eq. 19 we find that this is equivalent to the matrix $\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ which is the *NOT* gate in the X-basis. Similarly one can prove the ZX representation of the Y-, S- and CNOT gates. Projective (von Neumann) measurements are represented by spiders in the respective colour which are connected to a $k\pi$ spider indicating the measurement outcome m by $k = \frac{1-m}{2}$. Figure from [38]

Stacking multiple ZX diagrams *vertically* corresponds to a *tensor product* between the respective linear maps, while connecting them *horizontally* corresponds to *matrix multiplication*. This is illustrated by the construction of the Toffoli gate in the following example.

Example 3.1. (Construction of the Toffoli gate)

The *Toffoli (CCNOT-) gate* is a three-qubit gate that has two control qubits and one target qubit. It acts in the following way:

$$|a, b, c\rangle \mapsto |a, b, c \oplus (a \wedge b)\rangle \tag{21}$$

It can reproduce all reversible Boolean functions (AND, NOT, etc.) and together with the Hadamard and the phase gate, it constitutes a universal gate set.
The action of the Toffoli gate on the 3-qubit basis[8] can be expressed as the following matrix:

$$\text{TOF} := \begin{pmatrix} \mathbb{I}_6 & 0 \\ 0 & X \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \tag{22}$$

In words, we perform a logical AND between the first two qubits and XOR it with the last qubit. In order to construct the Toffoli gate in the ZX language, we first construct all necessary building blocks.

In ZX language we can express the 2-qubit AND map as a two-input one-output (non-Clifford) diagram with two generalized Hadamard gates:

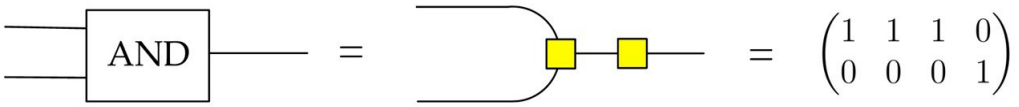


Figure 7: The 2-qubit AND map as a ZX diagram consisting of two generalized Hadamard gates and its matrix representation.

The AND matrix maps the input states $\{|00\rangle, |01\rangle, |10\rangle\}$ to $|0\rangle$ and the input state $|11\rangle$ to $|1\rangle$.
The XOR map can be expressed as a red spider with 2 inputs and 1 output:

Figure 8: The 2-qubit XOR map as a ZX diagram consisting of a single X-spider and its matrix representation.

The XOR matrix maps the input states $\{|00\rangle, |11\rangle\}$ to $|0\rangle$ and the input states $\{|10\rangle, |01\rangle\}$ to $|1\rangle$.

Another map we need for the construction of the Toffoli gate 'copies' computational basis states: $|0\rangle \mapsto |00\rangle$ and $|1\rangle \mapsto |11\rangle$.

[8] A basis for the three-qubit space is given by $\{|000\rangle, |001\rangle, |010\rangle, |100\rangle, |011\rangle, |110\rangle, |101\rangle, |111\rangle\}$

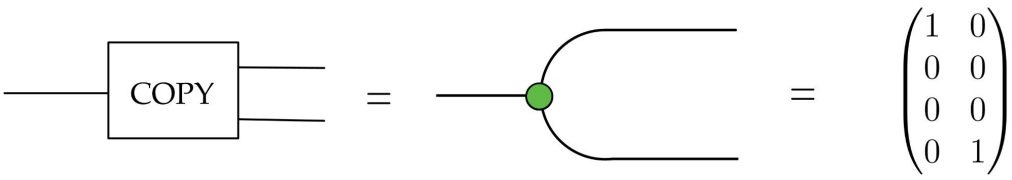


Figure 9: The 2-qubit COPY isometry as a ZX diagram consisting of a single Z-spider and its matrix representation.

And finally, the SWAP gate swaps the input qubits:

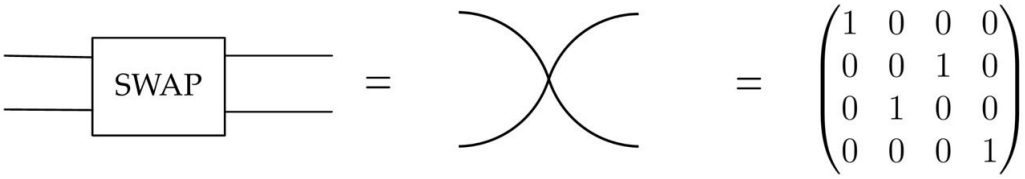


Figure 10: The 2-qubit SWAP gate as a ZX diagram consisting of two wires and its matrix representation.

We recall that the Toffoli gate is constructed by applying AND to the first two qubits, followed by XOR on the output and the third qubit:

Proposition 3.1. *The Toffoli gate has the following representation in the ZX calculus:*

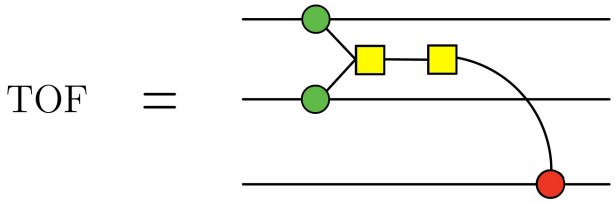


Figure 11: The 3-qubit Toffoli gate as a ZX diagram. The two generalized Hadamard gates perform the logical AND on the first two qubits before that output is XORed with the third qubit via the red X-spider.

Here the green spiders on the first two qubits act as a COPY isometry. They map $|0\rangle$ to $|00\rangle$ and $|1\rangle$ to $|11\rangle$.
Proof We can prove that this ZX diagram indeed performs the Toffoli gate on the input qubits by decomposing it into smaller parts and calculating their matrix representations.

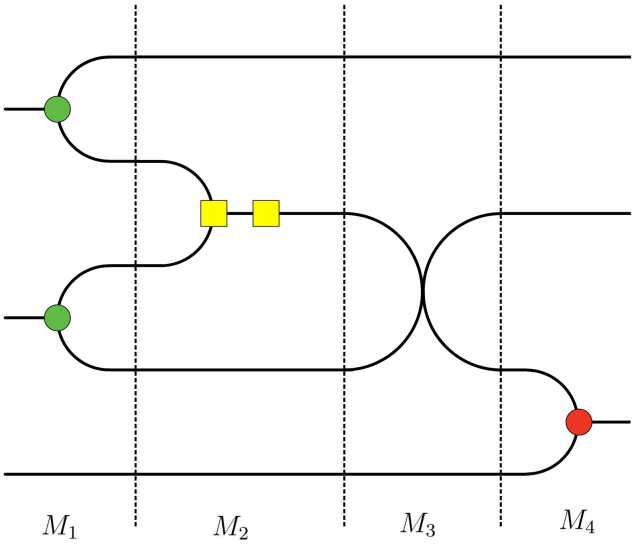


Figure 12: Decomposition of the Toffoli gate depicted in Fig. 11 into four sections, each of which can be described as a matrix tensor product.

In each section, the vertical stacking of lines corresponds to the tensor products of the respective matrices. The first section is the tensor product of two COPY isometries with the identity $\mathbb{1}$.

In the second section we have the tensor product of three identities with the AND map. Then follows a section with two identities and the SWAP gate. In the last section we calculate the tensor product of two identities and the XOR map.

$$M_1 = \begin{pmatrix} 1 & 0 \\ 0 & 0 \\ 0 & 0 \\ 0 & 1 \end{pmatrix}^{\otimes 2} \otimes \mathbb{1} \qquad M_2 = \mathbb{1} \otimes \begin{pmatrix} 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \otimes \mathbb{1}^{\otimes 2}$$

$$M_3 = \mathbb{1} \otimes \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \otimes \mathbb{1} \qquad M_4 = \mathbb{1}^{\otimes 2} \otimes \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

We then stack each section horizontally next to each other, which corresponds to matrix multiplication. The final expression for the matrix described by the ZX diagram in Fig. 12 is

$$M_4 \cdot M_3 \cdot M_2 \cdot M_1 = TOF \tag{23}$$

One can indeed verify directly that this expression simplifies to the Toffoli matrix given in Eq. 22

□

3.2 Rules of the ZX calculus

The property that makes ZX diagrams a powerful tool to create dynamic QECCs is that they can be arbitrarily deformed as long as the connectivity and the order of inputs and outputs of the entire diagram are preserved. It is possible to rewrite diagrams according to the following set of rules:

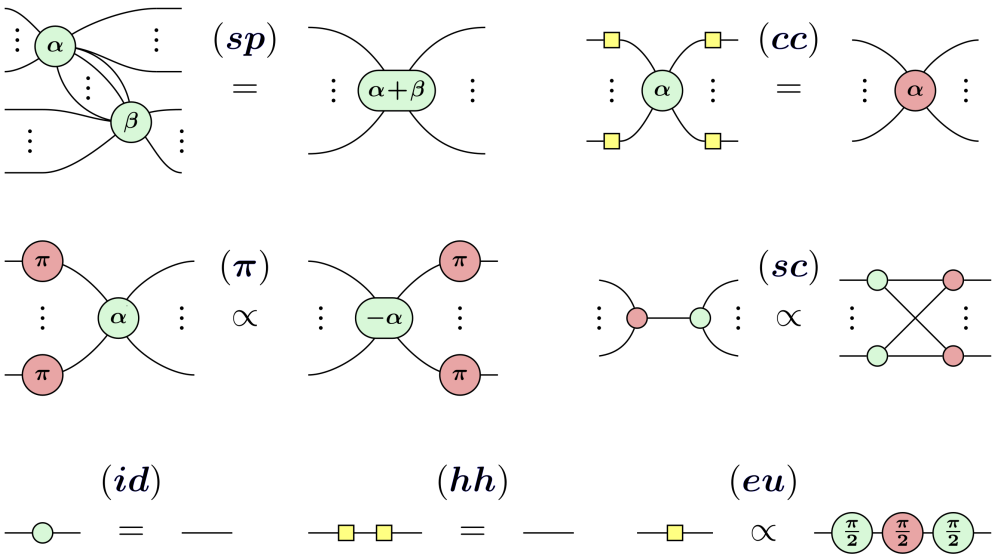


Figure 13: Rules to manipulate ZX diagrams, explained from top left to bottom right: (**sp**) Any spider can be unfused into multiple spiders where the respective phases are subtracted. Conversely, we can merge spiders of the same colour and add their phases. (**π**) When we *push* spiders with phase π through a spider of different colour its phase gets inverted. (**cc**) Whenever there are Hadamard boxes on all legs of a spider, they can be absorbed and the spider changes its colour. (**id**) Two legged phase-less spiders can always be omitted, (**hh**) just like a double Hadamard box. (**eu**) It is also possible to express the Hadamard boxes in terms of spiders. Figure from [38]

ZX diagrams together with the rewrite rules in Fig. 13 are a powerful tool to prove properties of states or unitaries and represent any quantum computation in the Clifford fragment of quantum mechanics[9]. In fact, by adding one more rule, it is possible to represent any true equality between linear maps in terms of ZX diagrams, meaning the ZX calculus is *complete* for standard quantum mechanics [47].

3.3 Pauli Webs

From this section on we restrict all ZX diagrams to be Clifford (see Def. 3.1). When the ZX calculus is used to represent quantum circuits, it offers a convenient method to track relevant properties of ZX diagrams, e.g. specific sets of

[9]The Clifford fragment of quantum mechanics is the subset of quantum theory restricted to applying Clifford operations (generated by the Hadamard gate, the S gate and CNOT) to computational basis states. It corresponds precisely to the subtheory captured by the stabiliser formalism, where Pauli operators are mapped to Pauli operators under conjugation. Interestingly, Clifford circuits are classically simulable via the *Gottesman-Knill theorem* (Theorem 10.7 in [3])

measurements. This is achieved by *Pauli webs* (introduced by Bombin et al. [45]), a graphical overlay created by highlighting edges between spiders in the diagram (as shown in Fig. 14) in red or green according to a set of rules from [38]:

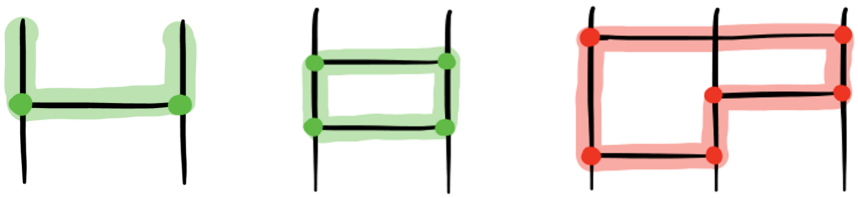


Figure 14: Some examples of Pauli webs. Figure from [37]

(r-i) A *Pauli spider* (spiders whose phase is an integer multiple of π) can have an even number of legs highlighted in its own colour **and/or** have all or none of its legs highlighted in the opposite colour.

(r-ii) A *Clifford spider* (Spiders whose phase is an integer multiple of $\frac{\pi}{2}$) can have either an even number of legs highlighted in its own colour and no legs in the opposite colour **or** have an odd number of legs highlighted in its own colour and all legs in the opposite colour.

(r-iii) If a Hadamard box is part of a Pauli web, its legs must always have different colours.

Introducing a Pauli web to a ZX diagram representing a quantum circuit can be thought of as adding π-spiders to the highlighted edges which have the same colour as the Pauli web. However, it must be possible to absorb those π-spiders into the spider connecting the edges according to the rules in Fig. 13 without altering its phase. An example of this can be seen in Fig. 15. From here on, if we relate two ZX diagrams to each other by " = ", that means they are equal *up to a nonzero scalar*.

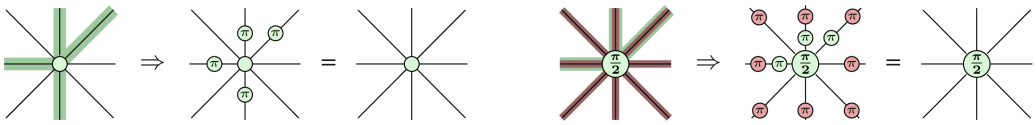


Figure 15: Highlighting edges to create a Pauli web corresponds to introducing π spiders to the respective edges such that they can be absorbed without changing the phases of the diagram. Figure from [38]

Using this ZX diagram decoration, we can keep track of detecting regions, stabilisers and logical operators which we introduce in the following sections.

3.3.1 Detectors and Detecting regions

Definition 3.2. *A **detector** is a set of measurements such that the product of all outcomes is **deterministic** under noiseless execution. If a detector is **violated** we can infer the occurrence of some **Pauli error** within the space-time region covered by the detector.*

In order to straightforwardly find detectors in our ZX diagrams, we introduce *detecting regions*.

Definition 3.3. ***Detecting regions** are Pauli webs where no input or output node of the diagram is highlighted. The **measurement spiders** in a detecting region that parametrise a red spider incident to a green highlighted edge, or a green spider incident to a red highlighted edge **constitute a detector**.*

Example 3.2. Again, we choose the $[\![4, 2, 2]\!]$ code as an example, where we repeatedly measure the two stabilisers of the code. The procedure described in the following is illustrated in Fig. 16. The first measurement outcome m_1 of the stabiliser $X_1X_2X_3X_4$ will yield a random outcome (as explained in Section 2.4.1). The following measurement outcome m_2 of the stabiliser $Z_1Z_2Z_3Z_4$ will also be random. However in all following measurements we will be in the common eigenspace of those two operators with eigenvalues set by the first two measurements. This means that the outcome m_3 of again measuring $X_1X_2X_3X_4$ will yield the same outcome as m_1 and the outcome m_4 of again measuring $Z_1Z_2Z_3Z_4$ will yield the same outcome as m_2. We can now define the formal products m_1m_3 and m_2m_4 as detectors since the multiplied measurement outcomes are deterministic. In fact, in the periodic $[\![4, 2, 2]\!]$ code, every product of measurement outcomes of the form $m_{i-2}m_i$ (for $i \geq 2$) will be a valid detector.

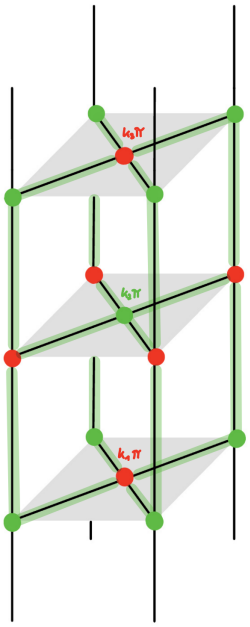


Figure 16: Periodic measurement of the 4 qubits in the $[\![4, 2, 2]\!]$ code in alternating bases (indicated by green (Z) and red(X)). The three measurement outcomes are indicated by $k_j = \frac{1-m_j}{2}$. The product of the (red) measurement outcomes m_1 and m_3 is deterministic and hence a valid detector.

These detectors enable us to detect any single-qubit error anywhere in the code. If, for example, the error X_1 occurred after the first stabiliser measurement of $Z_1Z_2Z_3Z_4$, the second measurement of this stabiliser will have the opposite result of the first. Hence the product of both measurements will be -1, indicating the occurrence of an error.

Finding multiple detecting regions in the ZX diagram of a code will allow us to reliably detect Pauli errors. Generally, a part of a detecting region in a specific colour can detect errors of the opposite colour. That is, a green Pauli web detects X- and Y-errors while a red Pauli web detects Z- and Y-errors. In the $[\![4, 2, 2]\!]$ code as depicted in Fig. 16, the green detecting region indicates that measurements m_1 and m_3 together form a detector.

3.3.2 Stabilisers and stabilising Pauli webs

Definition 3.4. *A **(co-)stabilising Pauli web** is a Pauli web where no input (output) edge of the diagram is highlighted and at least one output (input) edge is highlighted.*

Using the $[\![4, 2, 2]\!]$ code as an example again, one stabilising Pauli web is depicted in Fig. 17. There is a bijective correspondence between the π spiders on the outgoing edges introduced by a stabilising Pauli web and the stabilisers of a circuit (see Proposition A.3 in [38]). We see that the circuit in Fig. 17 stabilises the Pauli operator $Z_1Z_2Z_3Z_4$ because corresponding Z-spiders were introduced on the outgoing edges of the circuit by the stabilising region.

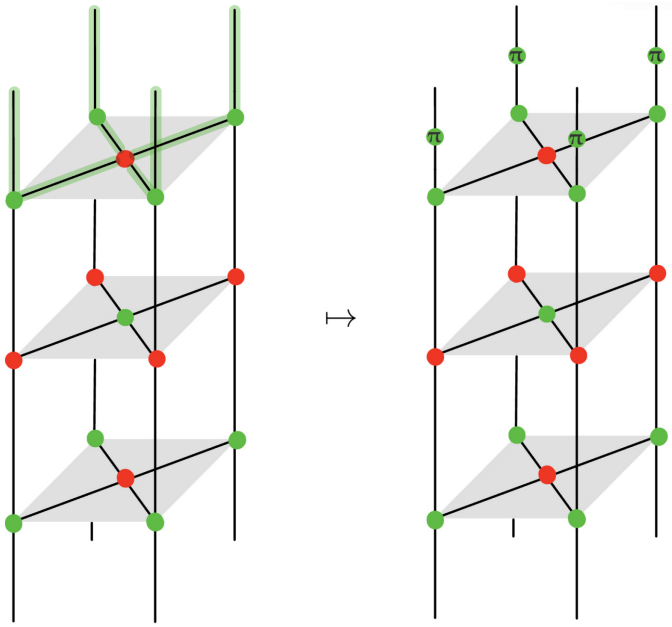


Figure 17: A stabilising Pauli web in the $[\![4, 2, 2]\!]$ code where no input edges of the diagram are highlighted. We can convert the green highlighted edges to π-spiders of the same colour and see that we end up with additional spiders that correspond to the code stabiliser $Z_1Z_2Z_3Z_4$.

3.3.3 Logicals and logical Pauli webs

The third class of Pauli webs helps us find logical operators on ZX diagrams representing circuits.

Definition 3.5. *A **logical Pauli web** is a Pauli web that is neither a detecting region nor a (co-)stabilising Pauli web.*

As a consequence, logical Pauli webs highlight at least one input and one output leg (see [37]). As with stabilising Pauli webs, we can find logical operators on the code by looking at the output edges of the code and observing what edges are highlighted. Again, the $[\![4, 2, 2]\!]$ serves as an example in Fig. 18. We see that only qubits 1 and 2 have output edges with green π spiders indicating that the operator Z_1Z_4 is a logical operator of the circuit. This is in accordance with the logical operator we initially found in the Hilbert space picture in Eq. 8.

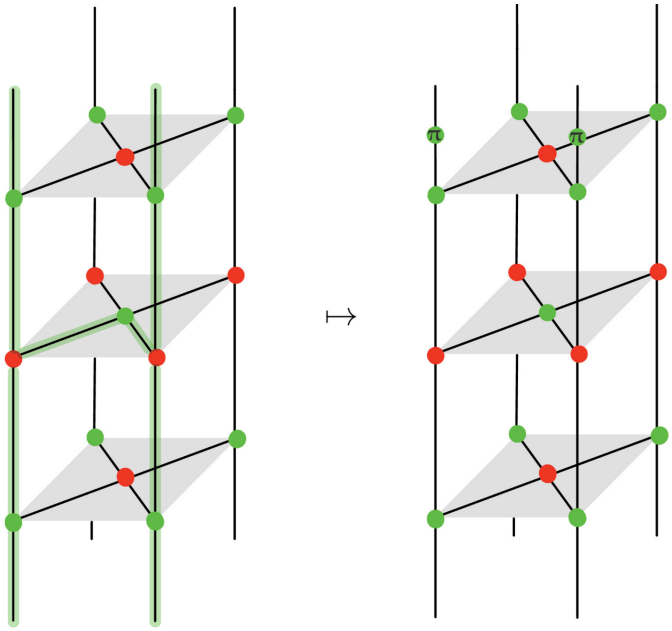


Figure 18: Logical Pauli webs highlight at least one input and one output leg. The highlighted edges can be converted to π spiders to find logical operators on the code. In this case we find the logical operator Z_1Z_4.

Finding multiple logical Pauli webs in the ZX diagram of a code will allow us to reliably detect *logical* errors, which is useful for verifying whether a decoder works successfully.

We can "close off" the logical Pauli web by including the qubit initialization and final measurements and thereby turning it into a detecting region. The set of deterministic measurement outcomes induced by the resulting Pauli web is what Stim interprets as an *observable*. If an observable is *flipped*, we know that a logical error occurred. Thus, introducing logical Pauli webs to the ZX diagram representing a QECC is necessary for a successful implementation of decoders like Chromobius (A.2.1).

After introducing the fundamental components of the ZX calculus, in the following section we will use the formalism to generate dynamic QECCs.

4 Floquetifying QEC Codes

The main goal of "Floquetifying" QEC codes is to lower measurement weight by decomposing multi-qubit parity checks into weight-one or weight-two measurements. We can apply several rewrites to the ZX diagram of a circuit to break high-weight measurements into smaller ones, at the cost of greater circuit depth.

In this section we introduce and analyse a naive approach before examining a more strategic Floquetification approach. But before that we remind the reader of the construction and properties of the QECC we want to Floquetify: the colour code.

4.1 Recap: Colour Codes

In this chapter we introduce and compare different Floquetifications of the 2D hexagonal colour code (henceforth the *vanilla colour code*). The main advantage of the colour code lies in efficient logical compilation. This means the Clifford gates H and S can be implemented transversally while the CNOT gate can be implemented using efficient lattice surgery methods [48–52]. We begin by revisiting the code's key features.

Colour codes are topological codes with qubits on the vertices v of a lattice that satisfies two conditions: (I) it is trivalent (three edges meet at every vertex), and (II) it is face three-colourable (plaquettes can be coloured red/blue/green so that no adjacent faces share a colour). Each face f (plaquette) hosts one stabiliser generator in each basis:

$$X_f = \prod_{v \in f} X_v, \qquad Z_f = \prod_{v \in f} Z_v. \tag{24}$$

making the code *CSS*[10]. The measurement basis of all plaquette stabilisers alternates between timesteps. Hence the code can be interpreted as an ISG code (see Def. 2.1) which is established for $t \geq 2$.
The smallest example of a colour code is the 7-qubit Steane code, which can be interpreted as a $d = 3$ colour code patch [53].
The number of encoded logical qubits is $k = n - \text{rank}(\mathcal{S})$ and can also be derived from the topology of the underlying lattice, which will be explained in the following.

When placed on an open surface, we refer to the code as *planar*. The simplest planar implementation uses a triangular patch where each boundary consists of faces in two of the three colours, not containing faces of the third colour. Due to the topological structure of the lattice, there exists only one independent *non-trivial cycle* connecting boundaries of different colour. This means we can encode one logical qubit. The weight of the shortest chain of X or Z operators supported on vertices connecting opposite boundaries, e.g. the smallest-weight logical operator defines the code distance d.

[10] A Calderbank–Shor–Steane (CSS) code has stabiliser generators that each consist solely of X- *or* Z-operators (and identity operators). In this way, bit-flip and phase-flip errors are detected independently.

The lattice can also be placed on a closed surface, e.g. a torus, to avoid boundary conditions. It is possible to derive the number of logical qubits k of a topological quantum code by relating it to the Euler characteristic χ.
On a closed orientable surface of genus g the Euler characteristic is $\chi = 2-2g$. A toric/surface code encodes $k = 2g$ logical qubits, and a 2D colour code encodes $k = 4g$ (equivalently, two copies of the toric code). Hence on a torus ($g = 1$, $\chi = 0$) a colour code encodes $k = 4$ logical qubits. For planar patches, k is determined by boundary conditions; for the triangular colour-code patch discussed above, $k = 1$.

Adding periodic boundary conditions by placing the lattice on a torus lets us define the code's stabilisers simply as the joint parity of all data qubits adjacent to a plaquette. The measurement schedule of the vanilla colour code can be expressed in the ZX language as depicted in Fig. 19

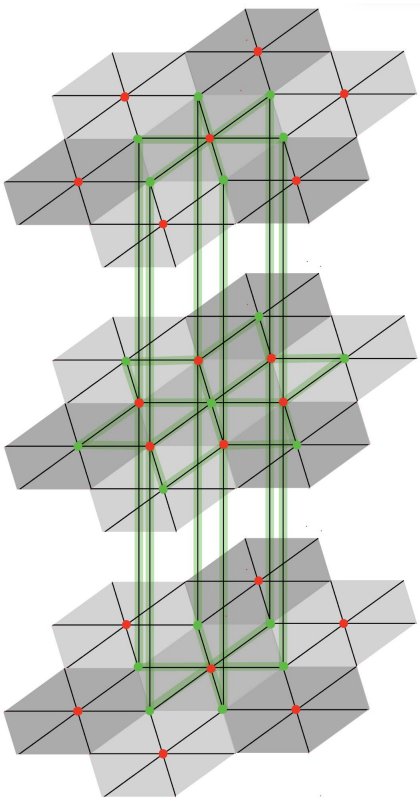


Figure 19: Three timesteps of the measurement schedule of the colour code as a ZX diagram. Plaquettes with a red X (green Z) spider represent weight-6 Z-(X-) measurements. Two consecutive measurements in one basis on the same plaquette form a detector (see Def. 3.2). The detecting region indicating the detector is given by the highlighted green edges of the diagram.

For the toric colour code, logical X/Z operators are homologically non-trivial coloured strings; there are two independent colours per non-contractible cycle, so on the torus we find 4 logical operators in total. Figure 20 shows a logical Z-operator on a colour code patch of size 6×6 (in terms of plaquettes) in its respective ZX diagram.

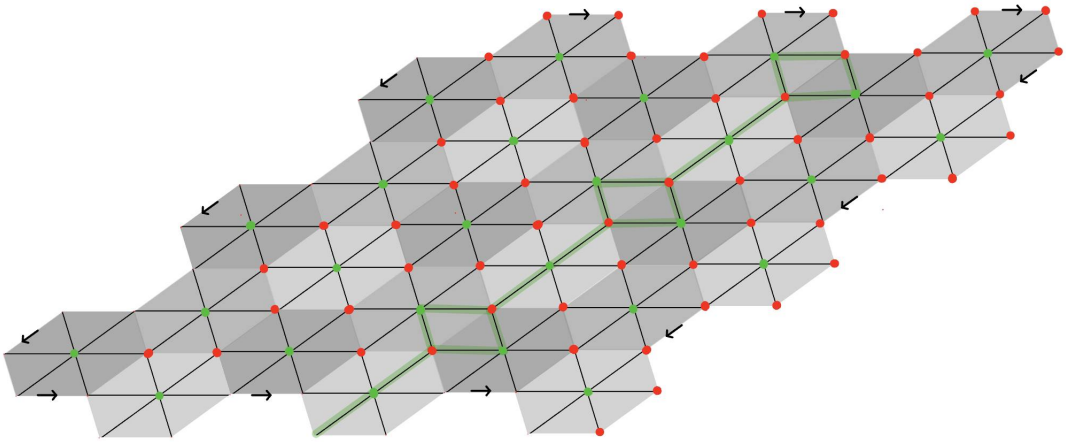
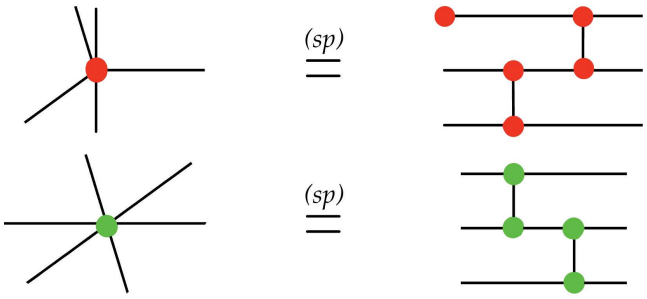


Figure 20: ZX diagram of a toric colour code patch; arrows mark the periodic edge identifications. The shown logical Pauli web wraps once around the torus and implements a logical Z-operator.

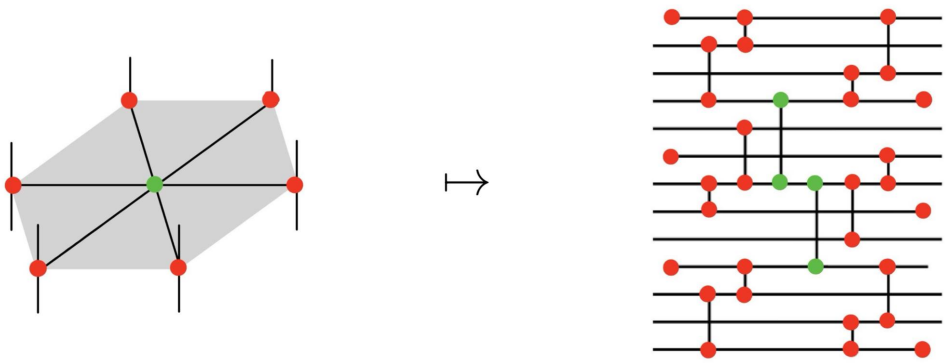
The code distance equals the weight of the shortest non-contractible logical operator that wraps once around the torus. Therefore, for a toric colour code with $L \times L$ plaquette stabilisers, the code distance is L.
Now we proceed by introducing the first Floquetification approach and applying it to the vanilla colour code.

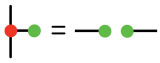
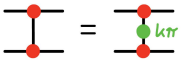
4.2 Naive Floquetification

Proposition 4.1. *By employing the spider-fusion rule (sp), we can make the following ZX-rewrites:*



Using these fragments we transform one weight-6 X-measurement ($X_1X_2X_3X_4X_5X_6$) using one ancillary qubit into 14 weight-two measurements and six weight-one measurements on 13 qubits:



Note that from now on, we depict single qubit nondestructive Pauli measurements as disconnected wires due to the following identity:  . Two-qubit measurements are drawn as  where we use *post-selection* and assume that $k = 0$.

We apply Proposition 4.1 to all plaquettes of the vanilla colour code depicted in Figure 20 to obtain the *naive Floquet colour code*.
As proposed by [37], we can reinterpret the qubit worldlines in such a way that their measurement schedule repeats every 13 timesteps on a periodic square lattice. In each timestep we can identify a periodic measurement pattern on the lattice whose unit cell contains 26 qubits. This superlattice has primitive vectors $\vec{v}_1 = \begin{pmatrix} 3 \\ 1 \end{pmatrix}$ and $\vec{v}_2 = \begin{pmatrix} -2 \\ 8 \end{pmatrix}$, as depicted in Figure 22. Note that this new 'unit cell' is not directly the Floquetified version of the hexagons from which the vanilla colour code is constructed. The plaquette's stabiliser measurement is spread over multiple timesteps in the new code. We can see that by looking at the second figure in Proposition 4.1, where one hexagonal vanilla colour code patch is spread over 8 timesteps during the Floquetification.

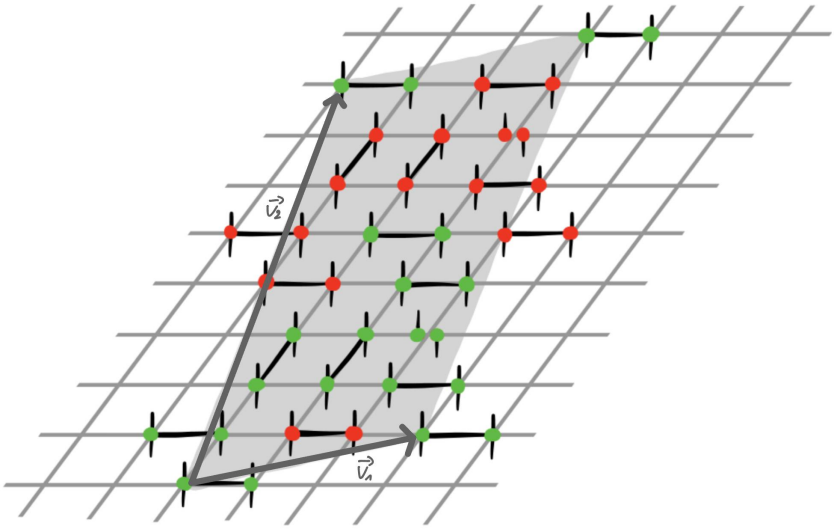


Figure 22: One periodic patch of the naively Floquetified colour code, spanned by $\vec{v}_1$ and $\vec{v}_2$.

These tiles are stitched together to generate the toric ZX diagram of the new code, interpreted as a measurement schedule of size L_x, L_y, and L_t. The measurement pattern at time step $t + 1$ is generated by shifting the pattern at t by $\vec{v}_{\text{shift}} = \begin{pmatrix} -1 \\ -3 \end{pmatrix}$. To describe the newly generated code, we define six parameters as follows: L_x and L_y give the number of tiles in horizontal and vertical direction, respectively, and L_t is the number of time steps the code is run. Empirically we find that Floquetification reduces the code's distance to $d = \frac{4}{3}\min(L_x, L_y)$. Hence the naively Floquetified colour code has the following code parameters (compare to Eq. 3).

$$[\![n = 26 \cdot L_x \cdot L_y,\ k = 4,\ d = \tfrac{4}{3}\min(L_x, L_y)]\!] \tag{25}$$

In order to evaluate the code's ability to protect logical information from noise, we have to find detecting regions and logical Pauli webs as defined in Def. 3.3 and Def. 3.5. This is achieved by translating the well-known detecting and logical Pauli webs from the vanilla code to the Floquetified version. In the vanilla colour code, the product of the outcomes of two consecutive plaquette measurements in the same basis is deterministic, as depicted in Figure 19. If a spider's legs are highlighted before it is unfused, the new edges generated by unfusion inherit the highlighting.

In this way, it is possible to find the equivalent set of measurements whose multiplied outcomes are deterministic in the Floquetified code. Note that the detecting region in the vanilla colour code consists of two measurements on six qubits over three time steps, while the new code's detecting region (found by Townsend-Teague et al. [37]) contains 16 measurements on 15 qubits over 8 time steps.

Similarly we see that the logical Pauli web of the code is increased in size: In the vanilla colour code on $L \times L$ plaquettes, the logical Z-operator spans around the torus parallel to the y-direction and over all time steps on $2 \cdot L + 1$ qubits (see Fig. 20). In the Floquetified code, it also covers all time steps and runs parallel to the new primitive vector $\vec{v}_1$ but touches $10 \cdot L_y$ qubits.
A section of the periodic logical Pauli web reduced to two timesteps and one patch is depicted in Figure 23.

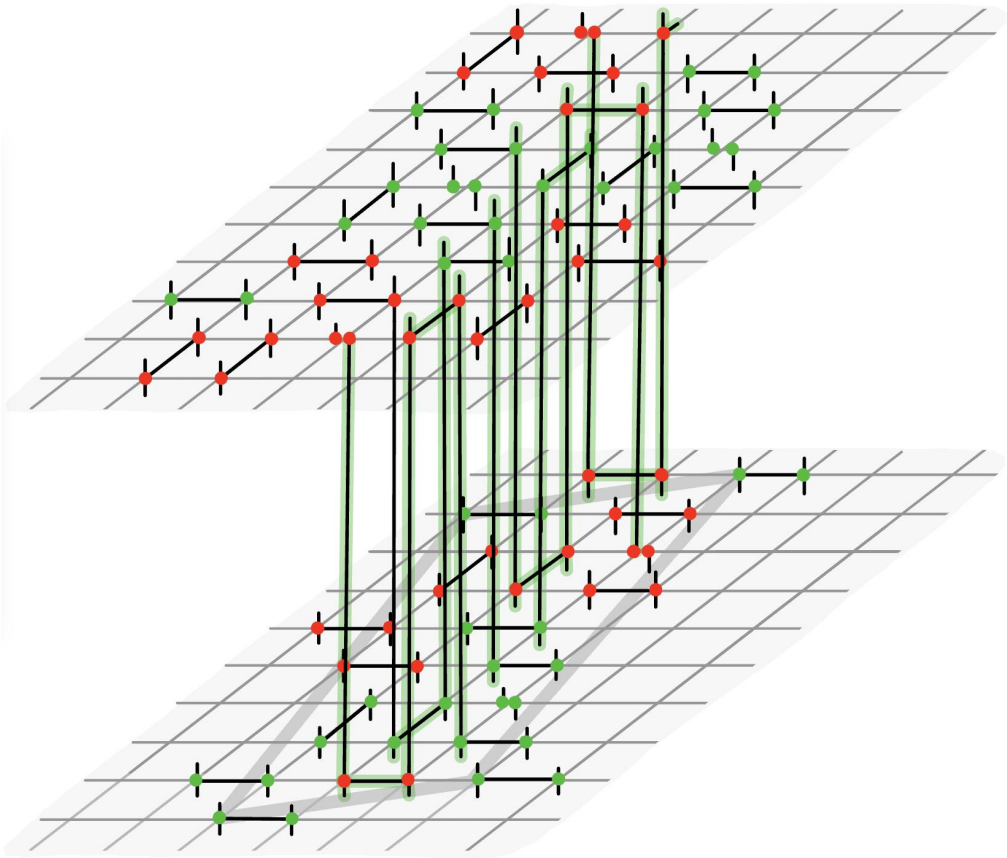


Figure 23: Section of the logical Pauli web representing the logical Z-operator. The Pauli web spans over all timesteps and in each timestep, it is contained in one straight string of lattice patches.

Having found detectors and a logical Pauli web, we implement the naive Floquet colour code in Stim and present our threshold simulation results in Section 5.2.

4.3 Fault-Equivalent Floquetification

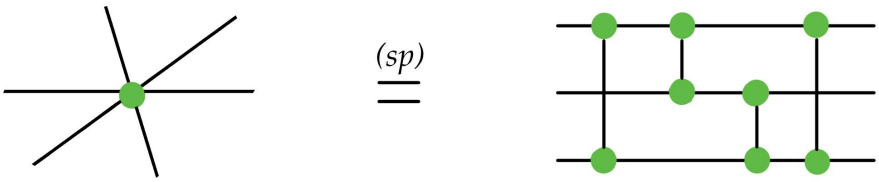
A major flaw of the naive Floquetification procedure is the decline of the code distance, because that means that even low-weight errors can cause intractable logical errors, as we will show in this section. To amend this, we introduce the notion of *fault-equivalence* as proposed by Ben Rodatz et al. [42] in a version slightly adjusted to our case:

4.3.1 Fault-Equivalence

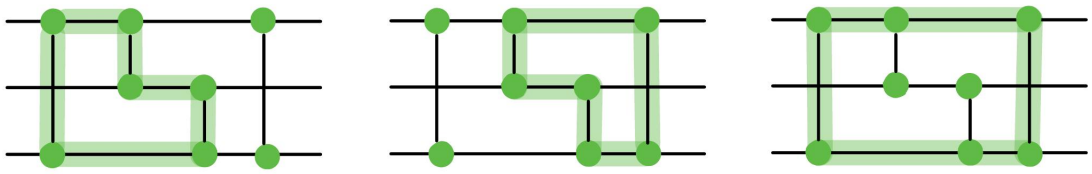
Definition 4.1. *Let A_1 and A_2 be two ZX diagrams representing quantum circuits, where A_2 is constructed by unfusing spiders (and thus adding edges) in A_1. The diagram A_2 is said to be **fault-equivalent** to A_1 if any Pauli error on a newly created edge in A_2 is either **detectable** or can be pushed to boundary edges **without increasing its weight**.*

In particular, Definition 4.1 implies that A_2 has at most the same distance as A_1, because the minimum-weight undetectable logical error on the first diagram will have at most the same weight on the second diagram. Since this weight is equal to the distance of the code represented by the ZX diagram, the distance is preserved when using *fault-equivalent rewrites*.

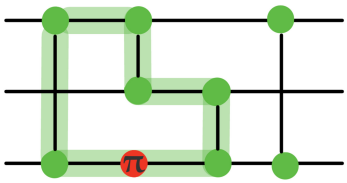
Proposition 4.2. *The following ZX diagrams are fault-equivalent.*



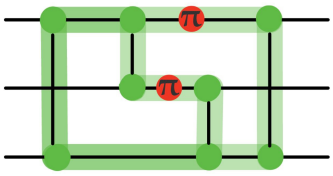
Proof We can find 3 detection webs in the right diagram, two of which are independent:



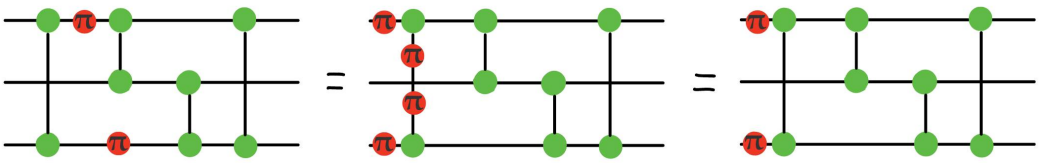
We can see immediately that every internal edge is part of at least one detector, meaning we can detect most odd-weight X errors, e.g.



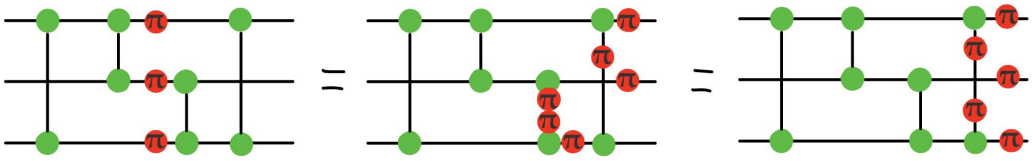
and even some even-weight errors, as long as not every detector contains an even number of errors:



The only type of weight-two error that is *un*detectable can be pushed to the boundary edges without increasing its weight:



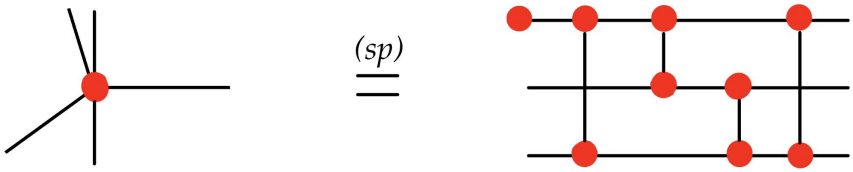
Similarly to the only type of undetectable weight-three error:



The two internal X-spiders in the last step will cancel, so the weight of the error remains the same.
Any other type of higher-weight error can either be detected or decomposed into the errors in the diagrams above.

□

Proposition 4.3. *The following ZX diagrams are fault-equivalent.*



Proof Any green (Z-)error on the newly added edge in the top left corner will not have an impact on the diagram, even though it is not part of a detector. This is because the first qubit is initialized in $|0\rangle$, which is an eigenstate of the

Z-operator. The rest of the diagram has the same properties as described in Proposition 4.2 and is therefore fault-equivalent.

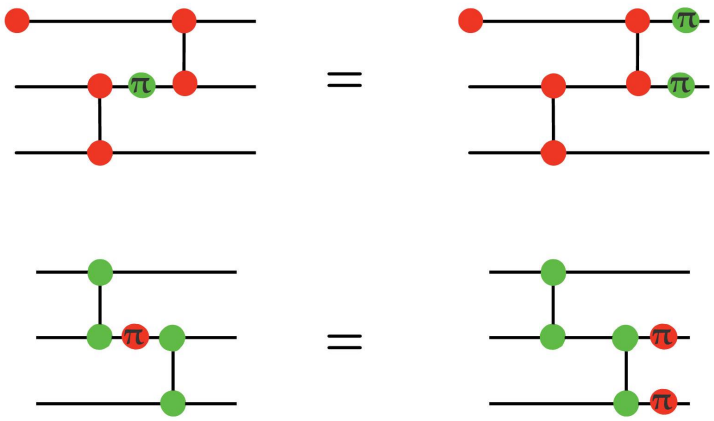
□

These are all the building blocks we need to Floquetify the colour code fault-tolerantly. But first, let's look at why the naive procedure from Proposition 4.1 does not fulfil that criterion.

4.3.2 Naive Floquetification is not Fault-Equivalent

Proposition 4.4. *The ZX-rewrites in Proposition 4.1 do not fulfil the criterion of fault-equivalence defined in Def. 4.1.*

Proof It is not possible to draw any valid detection region in the rewritten ZX diagram, meaning no error on a newly created edge is detectable. To show that the rewrites are not fault-equivalent according to Proposition 4.1, we have to show that errors on newly created edge *increase* in weight when pushed to the boundary edges. This can be seen in the following Figure for bit-/phase-flip errors between the two measurements:



Consequently, the rewrites from Proposition 4.1 are not fault-equivalent.

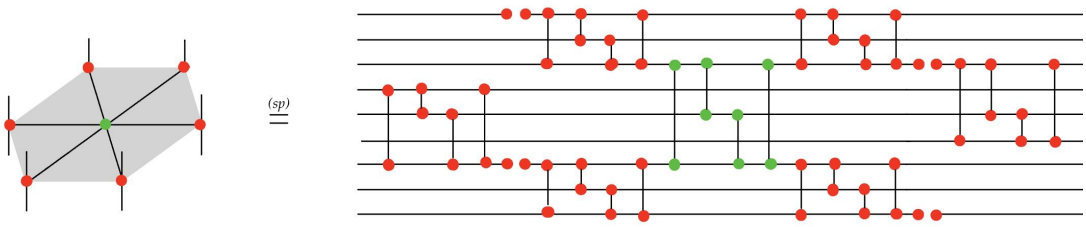
□

Since the code distance is not preserved, the naively Floquetified colour code cannot reliably detect larger-weight errors and we do not expect it to perform well under circuit-level noise simulations.
In the next section, we use a different set of rewrites to construct a fault-equivalent Floquet colour code.

4.4 Fault-equivalent Floquet Colour Code

In order to deconstruct the weight-six measurements of the colour code we use the following fault-equivalent rewrites:

Proposition 4.5. *The following ZX diagrams are fault-equivalent.*



Proof We can immediately identify in the figure above the building blocks that we know are fault-equivalent from Proposition 4.2 and Proposition 4.3. What remains to be shown is that the composition of fault-equivalent ZX diagrams is also fault-equivalent. This was proven by Ben Rodatz et al. in Proposition 3.11 in [42].

□

These rewrites can be applied to the two different kinds of spiders that appear in the vanilla colour code to generate a new periodic measurement pattern.

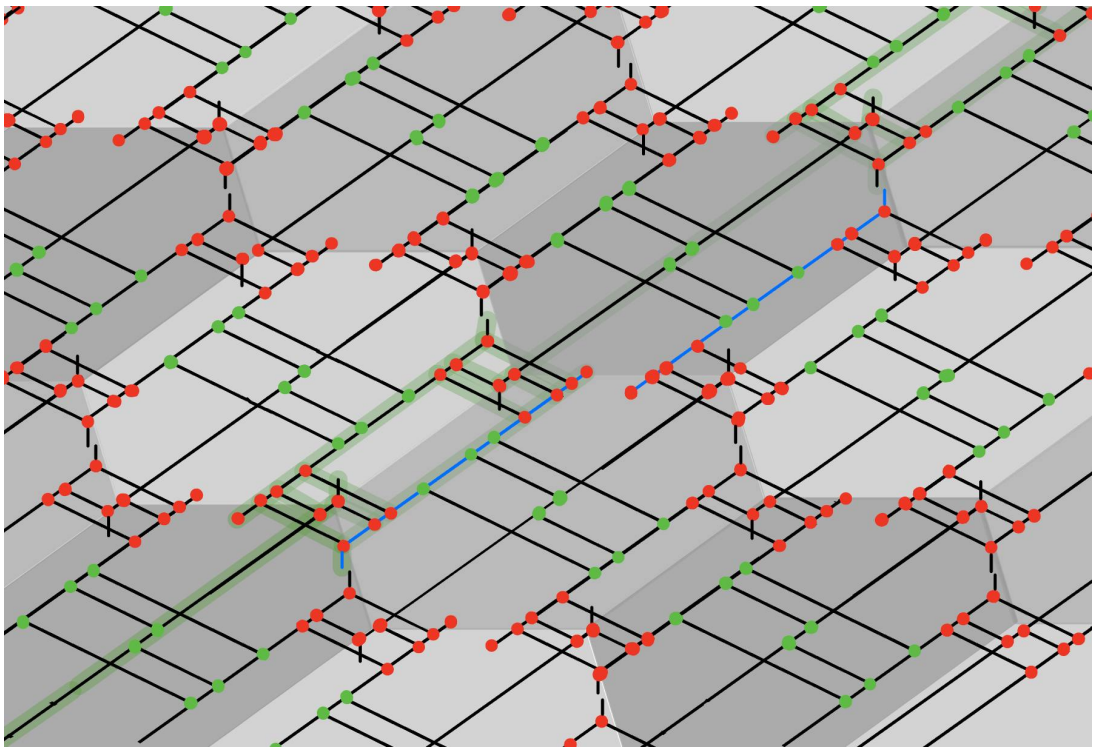


Figure 32: One layer of the intermediate representation towards the fault-equivalent Floquet colour code. This ZX diagram is generated by applying the fault-equivalent ZX rewrite from Prop. 4.5 to the ZX diagram representing the colour code in Fig. 20. The blue line is the newly assigned wordline of a qubit. Several of these ZX-diagrams are stacked vertically to deduce the new periodic measurement pattern. Qubit worldlines pointing 'upwards' are connected to qubit wordlines that are opened 'downwards' in the layer above. The green Pauli web represents a logical Z-operator, directly translated from the vanilla colour code.

Similarly, as in the naive Floquetification, by reinterpreting the qubit worldlines we see that each qubit is subject to a (time-)periodic measurement schedule consisting of 26 weight-1/2 measurements. However due to the placement of the qubits on a square lattice, idling time is added to the schedule, meaning the qubit measurement schedule repeats every 48 timesteps.

Within each timestep we find a (space-)periodic unit cell that contains 29 qubits and has primitive vectors $\begin{pmatrix}3\\1\end{pmatrix}$ and $\begin{pmatrix}-3\\15\end{pmatrix}$. By stitching $L_x \times L_y$ of these patches together we generate the desired toric lattice.

The detecting region of the vanilla code translates to a detecting region on nine qubits which contains 14 measurements over 29 timesteps on the fault-equivalent colour code. The significant increase in timesteps compared to the naively Floquetified colour code (29 vs 8) is due to the introduced idling time in the measurement schedule.

To find the logical Pauli web on the Floquetified code, we observed which measurements are highlighted in the intermediate diagram in Figure 32. One can see that there is a periodic pattern that repeats every second hexagon within each layer of the intermediate Floquetification and also on the same hexagon every second layer. For instance, the same type of measurement highlighted at $t = 3$ on qubits (3,25) and (2,26) is also highlighted in the same layer at $t = 31$ on qubits (1,26) and (0,27). One observes that a same type of measurement is highlighted again two layers above at timestep $t = 9$ on qubits (5,23) and (4,24).

This gives us sufficient information to determinate the periodicity of the new logical Pauli web:
If a measurement on qubits

$$[(x_0, y_0), (x_1, y_1)] \tag{26}$$

at $t = t_0$ is part of the logical Pauli web, then the measurements on qubits

$$[(x_0 + 2, y_0 - 2), (x_1 + 2, y_1 - 2)] \tag{27}$$

at $t = t_0 + 6$ and on qubits

$$[(x_0 - 2, y_0 + 1), (x_1 - 2, y_1 + 1)] \tag{28}$$

at $t = t_0 + 28$ are also part of the Pauli web.
To find the periodicity of that pattern in a *single* timestep, we have to solve the equation for the time difference between those measurements:

$$0 = 6a + 28b \qquad a, b \in \mathbb{Z} \tag{29}$$

$$\Rightarrow (a, b) = (-14k, 3k) \qquad k \in \mathbb{Z} \tag{30}$$

Inserting this solution into the qubit translations (Eq. 27 and Eq. 28) above, we find that

$$(x, y) \to (x - 14 \cdot 2 - 3 \cdot 2, y + 14 \cdot 2 + 3 \cdot 1) = (x - 34, y + 31) \tag{31}$$

So in each timestep, qubits (x, y) and $(x-34, y+31)$ undergo the same measurement that is part of the logical Pauli web of the Floquetified code. This newly found periodicity of the logical Pauli web is illustrated in Fig. 33

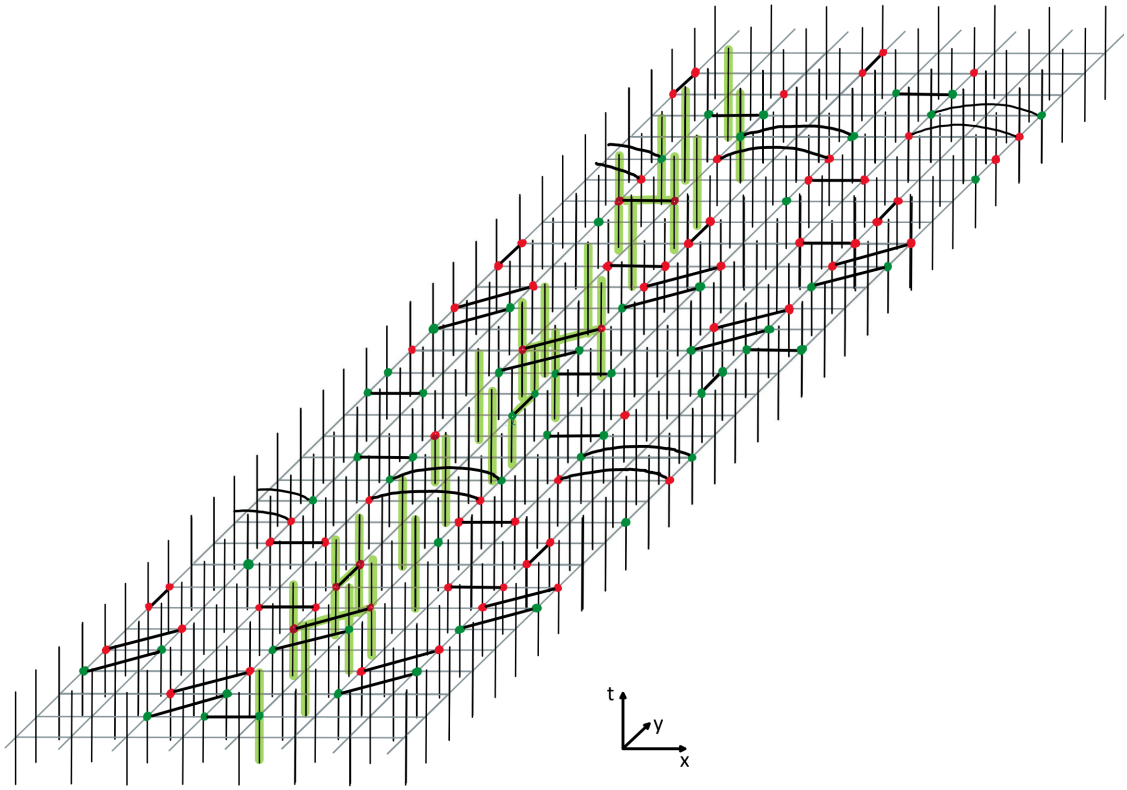


Figure 33: One period of the logical Pauli web of the fault-equivalent Floquet colour code in a single timestep.

We see that the "unit cell" of the logical operator in the Floquetified code is larger than the lattice's unit cell. Specifically, for the Z-logical that spans around the torus in the y-direction in Fig. 33, we need $(L_x = 3) \times (L_y = 3)$ lattice unit cells to support one full period of the logical Pauli web. Therefore not all code sizes in terms of $L_x \times L_y$ patched-together unit cells produce a valid logical Pauli web, which gives us the following constraint for the code size parameters:

$$(L_x \mod 3, \quad L_y \mod 3) \stackrel{!}{=} (0, 0) \tag{32}$$

Determining the code distance empirically is computationally expensive due to the large code size, both in time and space. However, using Stim, we find the following upper bound on the code distance:

$$d \leq \frac{4}{3} \min(L_x, L_y) \tag{33}$$

All code parameters apart from the space-like lattice sizes L_x and L_y and the number of timesteps L_t can be summarized as follows:

$$[\![n = 29 \cdot L_x \cdot L_y, \ k = 4, \ d = \tfrac{4}{3} \min(L_x, L_y)]\!] \tag{34}$$

Equipped with detectors and a logical Pauli web, we introduce circuit-level noise to the code and evaluate its resilience to errors in the following chapter.

5 Simulations

In this section we present numerical threshold simulations for three variants of the colour code. For all instances, we apply realistic circuit-level noise as described in Appendix A.4. We decode using the implementations from Craig Gidney et al. [54] and Joschka Roffe et al. [55] whose algorithms is described in Appendix A.2.

5.1 Vanilla Colour Code

First, we revisit the vanilla colour code to establish a baseline for the effectiveness of the Floquetification procedure. Under circuit-level noise, we find a threshold of about $p_{\text{th}} = (0.050 \pm 0.005)\%$:

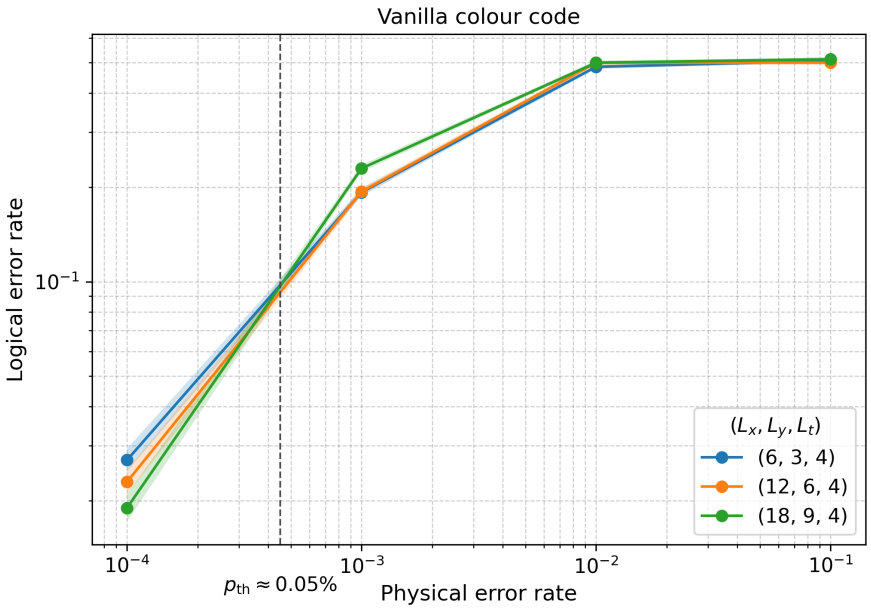


Figure 34: Threshold plot for the vanilla colour code using circuit-level noise as described in Appendix A.4. Data for codes consisting of $L_x \times L_y$ hexagons and L_t rounds of syndrome extraction. We find a threshold of $p_{\text{th}} = (0.050 \pm 0.005)\%$ which is lower than previously reported values. We attribute this to the absence of fault-tolerant syndrome extraction mechanisms in our circuits. However, this is acceptable because the aim of this simulation is the comparison of the unenhanced vanilla hexagonal colour code with our Floquetified versions.

For this simulation, we do not use fault-tolerant stabiliser measurement methods like Steane-style syndrome extraction [56]. This explains the discrepancy to previously found higher threshold results [57] that employ the same noise model.

5.2 Naive Floquet Colour Code

Next, we run simulations on the naively Floquetified colour code introduced in Section 4.2. As argued above (Prop. 4.4), the ZX rewrites to obtain this code were not fault-equivalent and thus did not preserve the original code distance. To monitor logical errors, we include one logical Z-operator to the code as shown in Figure 23.

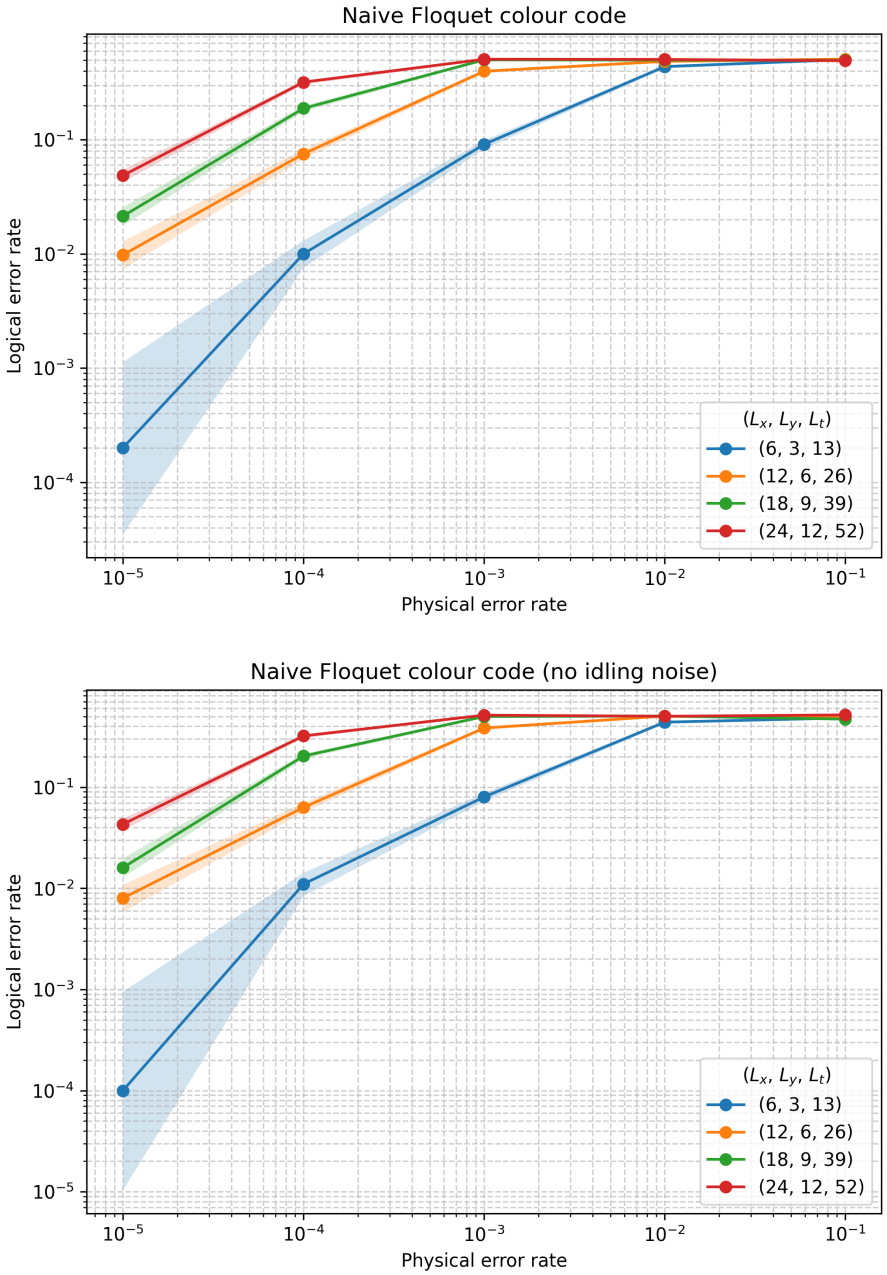


Figure 35: Threshold plot for the naive Floquet colour code using circuit-level noise (see Appendix A.4). Data for codes on square lattices consisting of $L_x \times L_y$ tiles as described in Figure 22 and L_t timesteps. To probe the role of idling, we compare runs with idling noise (top) and without (bottom). No significant impact is observed due to that error channel, likely because the naively Floquetified measurement schedule is comparatively dense. As expected, due to the code distance not being preserved, we see an increase in logical errors compared to the vanilla colour code and no threshold behaviour.

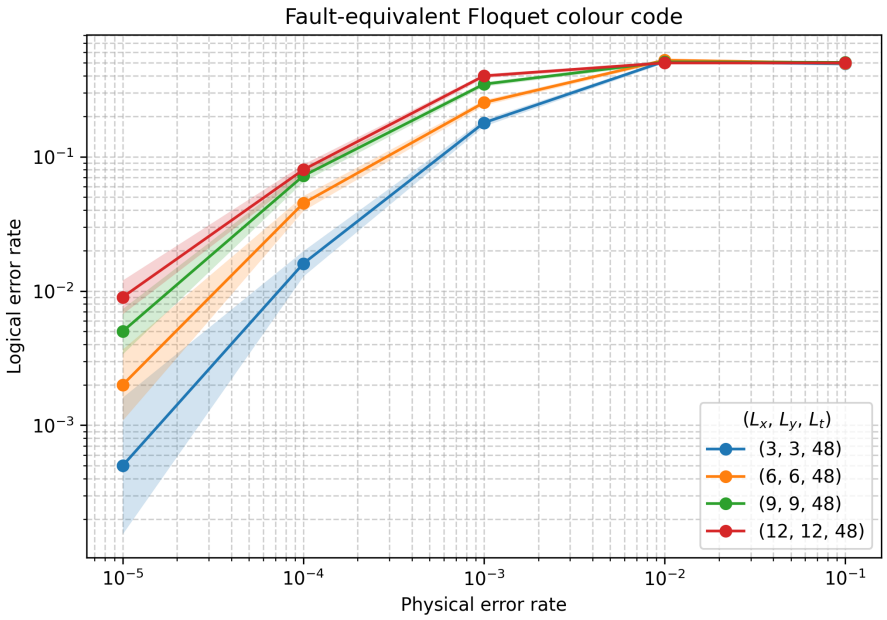
We do not observe *threshold behaviour*: increasing the code distance does not yield lower logical error rates for physical error rates below a specific p_{th}. For the plotted range of physical error probability, the curves do not cross and larger sizes remain above smaller ones. This can be attributed to the decreased distance,

because even low-weight errors can cause logical failures.

Additionally, as stated above, during the Floquetification, the number of qubits and measurements are increased significantly, resulting in larger detectors (see Ch. 3.3.1). A detector in the vanilla colour code contains 2 high-weight measurements over 3 timesteps. In the naively Floquetified code, a bulk detector contains 16 low-weight measurements over 8 timesteps. We expect large detectors to impair the decoder's ability to localise the underlying error [11]. This is because the detector can be violated by an error on any edge within the space-time region of the detector. If the detector density and consequently the detector overlap are low, the code becomes more susceptible to errors and decoding more challenging.

5.3 Fault-equivalent Floquet Colour Code

Next, we simulate the more promising Floquet colour code candidate from Section 4.4 obtained using fault-equivalent rewrites (see Def. 4.1). In particular, these rewrites are guaranteed to preserve the vanilla colour code's distance.



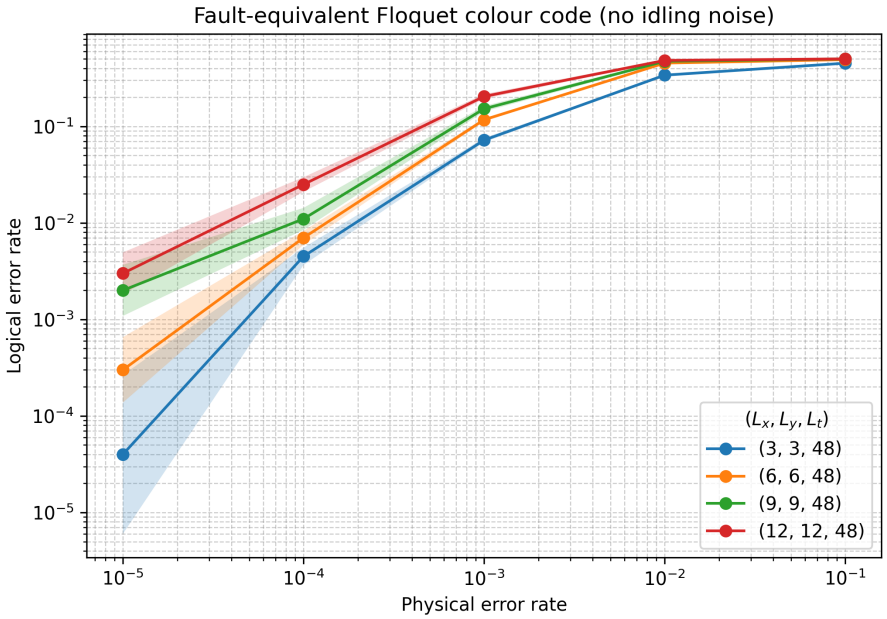


Figure 36: Threshold plot for the fault-equivalent Floquet colour code using circuit-level noise (see Appendix A.4). The data corresponds to codes on square lattices consisting of $L_x \times L_y$ 'unit cells' and L_t timesteps. To analyze the impact of idling errors on the code performance we compare simulation runs with idling noise (top) and without (bottom). As expected, the inevitable introduction of idling time increases the logical error rates by an order of magnitude. Although the code distance of the vanilla colour code is preserved, we do not observe threshold behaviour which we attribute to large detectors with low resolution and idling noise.

Contrary to our expectations, we observe similar behaviour to the naive Floquet code in Figure 35. The logical error rates improve slightly but we again do not observe threshold behaviour. For this code, idling noise has a significant impact on the performance, increasing the logical error rates by an order of magnitude. This is in accordance with our expectations as we are forced to introduce *idling time* to the qubits when placing them on a square lattice. For instance, in the Floquetified measurement schedule, qubits are not measured for up to 7 consecutive timesteps, increasing exposure to idling errors (see Def. A.12).

Another reason for the unexpectedly bad performance is the significant enlargement of the bulk detectors. A detecting region in the fault-equivalent colour code contains 14 measurements over 29 timesteps compared to 2 measurements over 3 timesteps in the vanilla code. There are many possible error locations that could cause the detector to be violated, decreasing the resolution of the detecting graph. This is likely to inhibit a successful decoding, because detector violations cannot be reliably allocated to the responsible Pauli errors.

6 Conclusion and Outlook

The goal of this thesis is to assess a new class of dynamic QECC. Floquet codes encode logical information dynamically and allow for non-commuting low-weight stabiliser measurements. This is desirable given hardware constraints in the qubit readout on most platforms. We examine various ZX rewriting methods to construct Floquet codes from the static CSS colour code. After graphically determining the measurement schedule as well as detecting regions and logical Pauli webs on the new codes, we perform threshold simulations using Stim.

The results we obtain do not meet our expectations, but they still provide valuable insights into the process of creating Floquet codes. Both simulations, using different sets of ZX rewrites do not exhibit threshold behaviour, meaning the logical error rate does not asymptotically decrease with the code size. We attribute this to the drastically increased size of detectors, the limited detector overlap and the impact of idling noise. By decreasing the measurement weight of the colour code stabilisers during Floquetification, we significantly increase the total number of measurements and the circuit depth.

However this raises various questions for future research endeavours: Can a different set of smaller and denser detectors improve the syndrome-to-error allocation of the decoder? For example, the detecting regions directly translated from the vanilla code might not form an optimal detector-generating set. Mathematically it is possible to find an equivalent set of detectors that have a finer resolution on the error-to-syndrome assignment. However this would increase the detector error model's size, making decoding computationally more expensive.

Is there a more efficient way to Floquetify CSS codes? The overall objective of Floquetification is reducing measurement weight such that QECCs with good code parameters can be run fault-tolerantly on physical platforms. However we see that unfusing spiders and stitching together the resultant ZX diagrams introduces an overhead in measurements and idling time. Thus, one objective could be to optimise Floquetification with respect to other parameters like qubit count and -connectivity or circuit depth. Circuits could also be adjusted to match a specific restricted gate set or qubit connectivity particular to a certain hardware.

Another useful direction would be the optimization of a *Floquetification program* that takes a static stabiliser code and a set of ZX rewrite rules as input and outputs a corresponding Floquet code, ideally tailored to a specific hardware. There exist already algorithms to find detection regions on ZX diagrams [58], which could be expanded to find logical Pauli webs as well. Such a tool would enable a more systematic and efficient search for promising candidates and facilitate the evaluation of new rewrite strategies. Also it would automate the detector/logical extraction from ZX diagrams that was performed 'by hand' in this thesis.

Future work could also include using the ZX calculus and Stim's detector error model to design a decoder that explicitly accounts for correlated space-time faults that arise from Floquet measurement schedules.

Appendix

A.1 Pure vs Mixed Quantum States

Analogous to classical bits with values 0 and 1, one can describe quantum states $|\psi\rangle$ in terms of the basis vectors $|0\rangle$ and $|1\rangle$ of a complex vector space $\mathbb{C}^2$. Quantum systems can be described using complex *Hilbert spaces* $\mathcal{H}$ which are complex vector spaces equipped with an inner product. $\mathcal{H}$ is complete w.r.t. the norm induced by the scalar product.

Definition A.1. ***Pure quantum states*** *can be described by normalized state vectors in a complex Hilbert space* $|\psi\rangle \in \mathcal{H}$.

Generally, any pure qubit state up to a global phase can be expressed as the superposition:

$$|\psi\rangle = \cos\frac{\theta}{2}|0\rangle + e^{i\phi}\sin\frac{\theta}{2}|1\rangle \tag{35}$$

This equation describes the geometrical representation of the general pure qubit state with parameters $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi]$ on a *Bloch sphere* depicted in the following Fig. 37

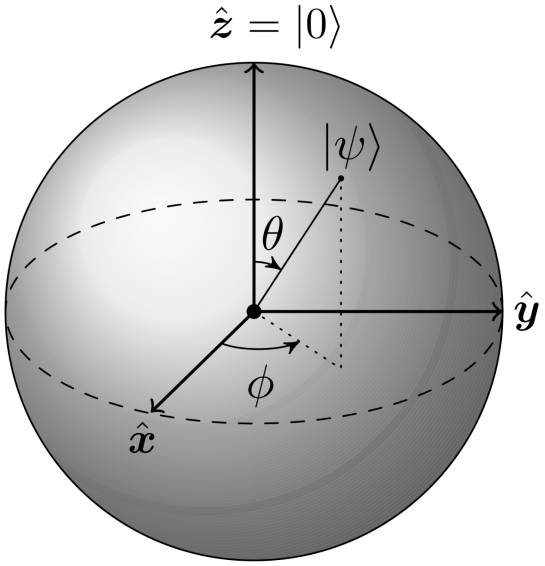


Figure 37: The state $|\psi\rangle$ of a qubit can be represented geometrically as a point on the Bloch ball.

However, this is not sufficient to describe all possible state preparations. Consider a situation where $|0\rangle$ is prepared with probability p_0 and $|1\rangle$ is prepared with probability $p_1 = 1 - p_0$. To describe this configuration, we have to resort to *density operators*.

Definition A.2. *A (general)* ***quantum state*** *of a d-dimensional quantum system is given by a* ***density matrix***

$$\rho = \begin{bmatrix} \rho_{00} & \rho_{01} & \cdots & \rho_{0,d-1} \\ \rho_{10} & \rho_{11} & \cdots & \rho_{1,d-1} \\ \vdots & \vdots & \ddots & \vdots \\ \rho_{d-1,0} & \rho_{d-1,1} & \cdots & \rho_{d-1,d-1} \end{bmatrix} \in \mathbb{C}^{d\times d},$$

which is positive semidefinite ($\rho \geq 0$) *and normalized* ($\mathrm{tr}(\rho) = 1$). *Its diagonal entries* $\{\rho_{ii}\}$ *form a classical probability distribution over the computational basis.*

Mixed states can be written as a probabilistic ensemble of pure states: $\rho = \sum_i p_i |\psi_i\rangle \langle\psi_i|$ with $p_i \geq 0$ and $\sum_i p_i = 1$; they represent classical uncertainty over pure quantum states.
A measure for the *purity* of a d-dimensional quantum state ρ is defined as $\mathrm{tr}(\rho^2) \in [1/d, 1]$. For example, the purity of a pure state $\rho = |0\rangle \langle 0|$ is

$$\mathrm{tr}\big((|0\rangle \langle 0|)^2\big) = \mathrm{tr}\left(\begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}\right) = 1 \tag{36}$$

while the state that is a completely random mixture of the two basis states $\rho = \frac{1}{2}(|0\rangle \langle 0| + |1\rangle \langle 1|)$ has a purity of

$$\mathrm{tr}\big((\tfrac{1}{2}(|0\rangle \langle 0| + |1\rangle \langle 1|))^2\big) = \mathrm{tr}\left(\begin{bmatrix} \frac{1}{4} & 0 \\ 0 & \frac{1}{4} \end{bmatrix}\right) = \tfrac{1}{2} \tag{37}$$

This is the *maximally mixed state*.
We find that in our geometrical interpretation (see Fig. 37), pure states lie on the surface of the Bloch ball while mixed states lie inside it.

A.2 Decoding Algorithms

In the following, we give the algorithms of the decoders used for the simulations in this thesis.

A.2.1 The Chromobius Decoder

The main difference between decoding 2D colour codes and standard matching decoders is the difference in syndrome weight. While single errors on many topological codes like the surface code produce a pair of syndromes, on the colour code, three stabiliser checks are triggered by a single error. To amend this, in standard decoding approaches, the colour code is projected onto three surface-code-like sublattices, according to the three colours of the hexagons. Each sublattice keeps only syndrome information associated with two of the three colours and can be decoded by MWPM before being recombined with the other two. The Chromobius decoder takes this a step further and uses the following algorithm:

Algorithm A.1 (Chromobius decoder)

Input: A QECC in the form of a Stim circuit that meets the requirements specified in [54]
Output: A bit $b \in \{0, 1\}$ revealing whether an observable has flipped or not.

initialization (once per circuit):

1. Generate the detector error model for the code and annotate all detectors with a basis $\{X, Z\}$ and a colour $\{r, g, b\}$
2. Split each detector into two "Möbius detectors" each belonging to one of three subgraphs:
   – *Bulk errors* become one edge in each subgraph
   – *Shift errors* become one edge in the two subgraphs that use that colour
   – *Composite errors* are decomposed into their bulk and shift components
3. Generate the matchable subgraphs whose vertices are Möbius detectors and whose edges are mapped error mechanisms

Online decoding (in each shot):

1. According to the set of activated detectors, activate the corresponding Möbius detectors in all subgraphs
2. Invoke MWPM on each subgraph to produce three disjoint sets of matched edges
3. Flatten the matching graph by projecting each Möbius edge back into the original graph by neglecting its subgraph property.
4. Drag each lifted detection event along an *Euler tour* on the connected edges. When encountering another detection event:
   – of the *same* colour → cancel both excitations
   – of *different* colour → fuse into an excitation of the third colour until no excitation remains
5. From the resulting excitation chain, infer whether logical observables have been flipped

It is not necessary to actually perform the physical recovery operation, but instead keep track of what observables were flipped due to the error. Using this information we can update the *Pauli frame* of the system and interpret future measurements accordingly.
There's a set of constraints the code has to fulfil, which are laid out in detail in [54].

A.2.2 Belief Propagation + Ordered Statistics Decoding

Belief propagation (BP) is a message-passing algorithm on the Tanner graph of the code [4]. In practice it is combined with ordered statistics decoding (OSD): First, BP supplies *soft decisions* and if it fails to find a syndrome-consistent error within a specific number of iterations, OSD post-processes the BP beliefs by selecting a reliable basis of columns in the parity-check matrix and solving the reduced system. A lightweight greedy search over low-weight configurations on the remaining columns ("combination sweep") further improves the suggested error candidate.

Algorithm A.2 (BP+OSD decoder)

Input: Parity-check matrices H_X, H_Z, channel error rates, measured syndrome $\vec{s}$, BP iteration cap $T_{\max}$, OSD search depth λ
Output: Estimated Pauli error e consistent with $\vec{s}$

Initialization (once per code):

1. Build two graphs: qubit nodes $\{q_i\}$ and check nodes from H_X (for Z-type syndromes) and H_Z (for X-type syndromes); connect q_i to checks that touch the qubit.
2. Set priors on each q_i from the noise model: $P(I), P(X), P(Y), P(Z)$ (depolarising) or $P(I), P(Z)$ on the H_X graph and $P(I), P(X)$ on the H_Z graph for CSS codes.

Online decoding (per syndrome):

1. Run BP on H_X and H_Z in parallel:
   a. Initialise variable-to-check messages with the priors.
   b. Iterate up to $T_{\max}$ with check updates (enforcing the measured stabiliser eigenvalue) and variable updates (multiplying prior and incoming check messages, then normalising).
   c. After each iteration, form qubit marginals and a candidate error e_{BP}; if it satisfies $H_X e_Z = s_X$ and $H_Z e_X = s_Z$ (mod 2), output e_{BP}.
2. If BP does not converge, invoke OSD separately on each CSS half:
   a. From the BP soft decisions, rank the bits to obtain an ordering $[O_{BP}]$ from most to least likely error.
   b. Reorder the columns of the parity-check matrix H by $[O_{BP}]$; pick the first rank(H) linearly independent columns as a basis $[S]$ (remainder $[T]$).
   c. Compute the OSD solution by inverting the full-rank submatrix $H_{[S]}$ to get $e_{[S]} = H_{[S]}^{-1} s$ and setting $e_{[T]} = 0$; map back to the original ordering to obtain e_{OSD}.

d. Greedy higher-order search (combination sweep): search all weight-one assignments on $[T]$ and all weight-two assignments on the first λ bits of $[T]$, using the analytic update $e_{[S]} = H_{[S]}^{-1}\big(s + H_{[T]}e_{[T]}\big)$. Keep the lowest-weight candidate e_{OSD} consistent with the syndrome.

3. Combine the X- and Z-decoding outputs into a Pauli operator and return it. If no candidate satisfies the syndrome, return failure.

The OSD post-processing step mitigates BP's failures due to degeneracy: it forces a full-rank basis, guarantees syndrome consistency, and the short greedy search often recovers the minimum-weight solution that BP alone misses.

A.3 Errors and Error Channels

As mentioned in the previous section, there are multiple ways to model noise when performing a quantum code simulation. In practice, the noise channel that is best suited to model physical errors depends strongly on the hardware platform. Therefore we will shortly explain common qubit implementations and define error channels that are suited to describe the most prominent noise phenomena on those setups.

A.3.1 Superconducting Qubits

Superconducting qubits are nonlinear LC resonators whose energy levels are unevenly spaced by adding a Josephson junction. The qubit state can then be manipulated by microwave pulses at the transition frequency ω_{01}. Due to dielectric loss in substrates and interfaces as well as quasiparticle tunneling, one observes *energy relaxation*. This means that the qubit decays from the excited state $|1\rangle$ to the ground state $|0\rangle$ over time. This effect can be described mathematically by the *amplitude damping channel*.

Definition A.3. *The **amplitude damping channel** describes the energy dissipation of a qubit where it decays from the excited state to the ground state $|1\rangle \to |0\rangle$ with probability γ. This can be expressed as the following non-unitary quantum channel:*

$$\mathcal{E}_{AD}(\rho) = E_0 \rho E_0^\dagger + E_1 \rho E_1^\dagger \tag{38}$$

with

$$E_0 = |0\rangle\langle 0| + \sqrt{1-\gamma}\,|1\rangle\langle 1|\,, \quad E_1 = \sqrt{\gamma}\,|0\rangle\langle 1|$$

E_1 changes a $|1\rangle$ state into a $|0\rangle$ state, corresponding to the loss of a quantum of energy. E_0 does not change $|0\rangle$, but reduces the state's amplitude. Physically, this means that the environment did not detect a photon emission from the qubit,

so the state was less likely to have been $|1\rangle$, hence its amplitude is suppressed.

Another relevant, uniquely quantum mechanical noise mechanism for superconducting qubits is the loss of coherence between the states $|0\rangle$ and $|1\rangle$ which arises from magnetic impurities in the hardware or charge noise on the chip's surface. This can be modelled as the *phase flip channel* or the *dephasing channel*.

Definition A.4. *The **dephasing** or **phase damping channel** introduces phase decoherence between the states $|0\rangle$ and $|1\rangle$ and can be described by*

$$\mathcal{E}_{PD}(\rho) = E_0 \rho E_0^\dagger + E_1 \rho E_1^\dagger \tag{39}$$

with

$$E_0 = |0\rangle\langle 0| + \sqrt{1-\gamma}\,|1\rangle\langle 1|, \quad E_1 = \sqrt{\gamma}\,|1\rangle\langle 1|$$

As for amplitude damping, E_0 leaves $|0\rangle$ unchanged but scales the amplitude of $|1\rangle$ by $\sqrt{1-\gamma}$. E_1 projects onto $|1\rangle$ with weight γ. The combined channel damps off-diagonal elements and thus reduces coherence without energy exchange.
The phase-flip channel can be interpreted as a *Pauli-twirled* [11] approximation of the dephasing channel.

Definition A.5. *In the **phase-flip channel**, a Z-error is applied with probability p:*

$$\mathcal{E}_Z(\rho) = (1-p)\rho + pZ\rho Z \tag{40}$$

or expressed as a matrix acting on the density operator:

$$\rho = \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} \longmapsto \begin{pmatrix} \rho_{00} & (1-2p)\rho_{01} \\ (1-2p)\rho_{10} & \rho_{11} \end{pmatrix}$$

Analogously the phase-flip channel, where a qubit is subject to a Z-error, it is possible that an X-error happens on a qubit.

[11] Pauli-twirling converts arbitrary quantum channels into *Pauli channels* which solely consist of Pauli operators. This is achieved by conjugating a quantum channel by randomly chosen single-qubit Pauli operators and averaging over them. Since Pauli operators are self-conjugate, they are mapped to themselves, but different Pauli operators pick up different signs. When summing over many arbitrary Pauli conjugations, any mixing terms between distinct Pauli operators cancel. As a result, we get a Pauli channel that has the same action (on average) over Pauli eigenstates.[59]

Definition A.6. *In the **bit-flip channel**, an X-error is applied with probability p:*

$$\mathcal{E}_X(\rho) = (1-p)\rho + pX\rho X \tag{41}$$

or expressed as a matrix acting on the density operator:

$$\rho = \begin{pmatrix} \rho_{00} & \rho_{01} \\ \rho_{10} & \rho_{11} \end{pmatrix} \longmapsto \begin{pmatrix} (1-p)\rho_{00} + p\,\rho_{11} & (1-p)\rho_{01} + p\,\rho_{10} \\ (1-p)\rho_{10} + p\,\rho_{01} & (1-p)\rho_{11} + p\,\rho_{00} \end{pmatrix}$$

A.3.2 Trapped ion Qubits

Trapped ion qubits consist of a single atomic ion with internal electronic levels. Due to the hyperfine structure of the ion's energy levels, a qubit can be created by exciting a high energy level using microwave radiation or Raman transitions. The qubit readout is conducted by observing the state-dependent fluorescence of the ion using a photomultiplier.
Other than using the dephasing channel to describe magnetic field fluctuations that influence the energy level splitting, it is necessary to describe errors that affect entangling gates. This can happen when multiple qubits share a common vibrational mode and are coupled simultaneously by environmental noise. These *correlated errors* are described by multi-qubit Kraus operators.

Definition A.7. *For a n-qubit system a noise channel $\mathcal{E}$ is **uncorrelated** if it decomposes into single-qubit channels:*

$$\mathcal{E} = \mathcal{E}_1 \otimes \mathcal{E}_2 \otimes \cdots \otimes \mathcal{E}_n$$

*A noise process is **correlated** if it is not uncorrelated.*

Physically this means that the environment couples simultaneously to multiple qubits.

A.3.3 Photonic Qubits

It is possible to use photons as qubits and interpret different polarisations as qubit states. Since photons barely interact with each other physically, beam splitters are used to enforce interference between the particles. By measuring a subset of photons, the others will end up entangled. The major source of errors is the loss of photons which is modelled mathematically by the *erasure channel*.

Definition A.8. *The **erasure channel** has two inputs 0 and 1 and three outputs, 0, 1 and e. With probability $1-p$ the input is not changed, with probability p the input is erased and replaced by e. This can be expressed in terms of Kraus operators:*

$$\mathcal{E}_E(\rho) = E_0\rho E_0^\dagger + E_1\rho E_1^\dagger + E_2\rho E_2^\dagger \tag{42}$$

with

$$E_0 = \sqrt{1-p}\mathbb{I}, \quad E_1 = \sqrt{p}\,|e\rangle\langle 0|, \quad E_2 = \sqrt{p}\,|e\rangle\langle 1|$$

A.4 Theoretical Noise Models

The error channels from the previous section can be combined to realistically model noisy environments in simulations. In this section we will go over different noise models and explain which model we chose for our simulations and why. The simplest noise model assumes that qubits are subject to a bit-flip, phase-flip or both errors, with probability p respectively.

Definition A.9. *The **single-qubit depolarising channel** assumes a combination of the bit-flip and phase-flip error channel. The error operator on the right is applied to the single-qubit quantum state $\rho \in \mathbb{C}^2$ with the given probability.*

$$\text{DEP1}(p): \quad \begin{aligned} 1-p &\rightarrow \mathbb{1} \\ p/3 &\rightarrow X \\ p/3 &\rightarrow Y \\ p/3 &\rightarrow Z \end{aligned}$$

That noise channel can be extended to act on two qubits:

Definition A.10. *The **two-qubit depolarising channel** assumes a correlated bit-flip and phase-flip error channel acting on two qubits simultaneously. The error operator on the right is applied to the two-qubit quantum state $\rho \in (\mathbb{C}^2)^{\otimes 2}$ with the given probability.*

$$\text{DEP2}(p): \quad \begin{array}{llll} 1-p \rightarrow I\otimes I & p/15 \rightarrow I\otimes X & p/15 \rightarrow I\otimes Y & p/15 \rightarrow I\otimes Z \\ p/15 \rightarrow X\otimes I & p/15 \rightarrow X\otimes X & p/15 \rightarrow X\otimes Y & p/15 \rightarrow X\otimes Z \\ p/15 \rightarrow Y\otimes I & p/15 \rightarrow Y\otimes X & p/15 \rightarrow Y\otimes Y & p/15 \rightarrow Y\otimes Z \\ p/15 \rightarrow Z\otimes I & p/15 \rightarrow Z\otimes X & p/15 \rightarrow Z\otimes Y & p/15 \rightarrow Z\otimes Z \end{array}$$

If one simply wants to simulate readout errors, i.e. the inversion of a measurement result, this is imitated by the following noise model:

Definition A.11. *When performing a projective measurement on a number of qubits we get as a result the eigenvalue $m \in \{\pm 1\}$ of the respective projector M. If a **measurement error** occurs, this result is flipped:*

$$\text{MERR}_M(p): \quad \begin{aligned} 1-p &\rightarrow M_m \\ p &\rightarrow M_{-m} \end{aligned}$$

The most realistic noise model assumes a combination of multiple error channels to closely mimic the physical noise situation. To account for an error in the initial state of the system, we use *initialisation noise* to mimic a depolarising error at the beginning of the circuit. Whenever we apply gates to our system we assume them to be faulty, meaning we apply depolarising noise right after the

gate's execution. When a qubit is not subject to a measurement or a gate in a timestep, we imitate its state decoherence by applying a depolarising channel in that timestep. Additionally, we assume all measurements to be faulty with a specific probability. This is combined in the concept of *circuit level noise*.

Definition A.12. ***Circuit-level noise*** *is a quantum channel that combines* ***initialisation noise***, ***idling noise***, *single- and two-qubit* ***gate errors*** *as well as* ***measurement noise***. *The respective noise channels with probability p are implemented using depolarising noise and measurement noise:*

Noise phenomenon	**Channel**
Idle	$\mathrm{DEP1}(p)$
$\big($any single-qubit Clifford$\big)\, U_1$	$\mathrm{DEP1}(p) \cdot U_1$
$\big($any two-qubit Clifford$\big)\, U_2$	$\mathrm{DEP2}(p) \cdot U_2$
Initialisation	$\mathrm{DEP1}(p) \cdot \rho_{t=0}$
M_P	$\mathrm{DEP1}(p) \cdot \mathrm{MERR}_P(p)$

References

[1] Daniel Gottesman. *Stabilizer Codes and Quantum Error Correction*. PhD thesis, California Institute of Technology, 1997.

[2] Daniel Gottesman. *An Introduction to Quantum Error Correction and Fault-Tolerant Quantum Computation*. arXiv: 0904.2557, 2009.

[3] Isaac L. Chuang Michael A. Nielsen. *Quantum Computation and Quantum Information*. Cambridge: Cambridge University Press, 2010.

[4] Joschka Roffe. *Quantum Error Correction: An Introductory Guide*. arXiv: 1907.11157, 2019.

[5] Peter W. Shor. *Scheme for reducing decoherence in quantum computer memory*. Physical Review A, 52(4), 1995.

[6] Andrew M. Steane. *Error Correcting Codes in Quantum Theory*. Physical Review Letters, 77(5), 1996.

[7] Dorit Aharonov and Michael Ben-Or. *Fault-Tolerant Quantum Computation with Constant Error Rate*. Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing, pages 176–188, 1997.

[8] Peter W. Shor A. R. Calderbank. *Good Quantum Error-Correcting Codes Exist*. arXiv: quant-ph/9512032, 1995.

[9] S. B. Bravyi and A. Y. Kitaev. *Quantum codes on a lattice with boundary*. arXiv: quant-ph/9811052, 1998.

[10] A. G. Fowler. *Surface codes: Towards practical large-scale quantum computation*. Physical Review A, 86, 2012.

[11] D. Litinski. *A Game of Surface Codes: Large-Scale Quantum Computing with Lattice Surgery*. arXiv: 1808.02892, 3:128, 2019.

[12] Nicolas Delfosse. *Decoding color codes by projection onto surface codes*. arXiv: 1308.6207, 2013.

[13] D. S. Wang et al. *Threshold error rates for the toric and surface codes*. arXiv: 0905.0531, 2009.

[14] R. Acharya, L. Aghababaie-Beni, I. Aleiner, and collaborators. *Quantum error correction below the surface code threshold*. arXiv: 2408.13687, 2024.

[15] I. Besedin and collaborators. *Realizing lattice surgery on two distance-three repetition codes with superconducting qubits*. arXiv: 2501.04612, 2025.

[16] N. Lacroix and collaborators. *Scaling and logic in the colour code on a superconducting quantum processor*. Nature, page 1, 2025.

[17] I. Pogorelov, F. Butt, L. Postler, C. D. Marciniak, P. Schindler, M. Müller, and T. Monz. *Experimental fault-tolerant code switching.* Nature Physics, 21:298, 2025.

[18] S. Dasu, S. Burton, K. Mayer, D. Amaro, J. A. Gerber, K. Gilmore, D. Gresh, D. DelVento, A. C. Potter, and D. Hayes. *Breaking even with magic: demonstration of a high-fidelity logical non-Clifford gate.* arXiv: 2506.14688, 2025.

[19] D. Bluvstein and collaborators. *Architectural mechanisms of a universal fault-tolerant quantum computer.* arXiv: 2506.20661, 2025.

[20] Micheal James. *Quantum Error Correction and Decoherence.* ResearchGate, 2025.

[21] David P. DiVincenzo and Firat Solgun. *Multi-qubit parity measurement in circuit quantum electrodynamics.* arXiv: 1205.1910, 2012.

[22] Matthew B. Hastings and Jeongwan Haah. *Dynamically Generated Logical Qubits.* Quantum; arXiv: 2107.02194, 5:564, 2021.

[23] C. Vuillot. *Planar Floquet Codes.* arXiv: 2110.05348, 2021.

[24] P. Hilaire, T. Dessertaine, B. Bourdoncle, A. Denys, G. de Gliniasty, G. Valentí-Rojas, and S. Mansfield. *Enhanced Fault-tolerance in Photonic Quantum Computing: Floquet Code Outperforms Surface Code in Tailored Architecture.* arXiv: 2410.07065, 2024.

[25] D. Aasen, Z. Wang, and M. B. Hastings. *Adiabatic paths of Hamiltonians, symmetries of topological order, and automorphism codes.* Physical Review B, 106(8):085122, 2022.

[26] Bob Coecke and Ross Duncan. *Interacting Quantum Observables.* In *Automata, Languages and Programming*, pages 298–310. Springer, 2008.

[27] Bob Coecke and Ross Duncan. *Interacting quantum observables: categorical algebra and diagrammatics.* arXiv: 0906.4725, 13:043016, 2011.

[28] Aleks Kissinger. *Phase-free ZX diagrams are CSS codes: (...or how to graphically grok the surface code).* arXiv: 2204.14038, 2022.

[29] M. Backens, H. Miller-Bakewell, G. de Felice, L. Lobski, and J. van de Wetering. *There and back again: A circuit extraction tale.* Quantum, 5:421, 2021.

[30] Ross Duncan and Simon Perdrix. *Rewriting measurement-based quantum computations with generalised flow.* In *International Colloquium on Automata, Languages, and Programming*, pages 285–296. Springer, 2010.

[31] Aleks Kissinger and John van de Wetering. *Universal MBQC with generalised parity-phase interactions and Pauli measurements.* Quantum, 3, 2019.

[32] N. de Beaudrap, X. Bian, and Q. Wang. *Fast and effective techniques for T-count reduction via spider nest identities.* arXiv: 2004.05164, 2020.

[33] R. Duncan, A. Kissinger, S. Pedrix, and J. van de Wetering. *Graph-theoretic Simplification of Quantum Circuits with the ZX-calculus.* Quantum, 4:279, 2020.

[34] A. Kissinger and J. van de Wetering. *Reducing the number of non-Clifford gates in quantum circuits.* Physical Review A, 102:022406, 2020.

[35] J. Huang, S. M. Li, L. Yeh, A. Kissinger, M. Mosca, and M. Vasmer. *Graphical CSS code transformation using ZX calculus.* In *Proceedings of the Twentieth International Conference on Quantum Physics and Logic, Paris, France, 17-21st July 2023*, volume 384, pages 1–19. Open Publishing Association, 2023.

[36] A. Kissinger and J. van de Wetering. *Scalable spider nests (...or how to graphically grok transversal non-clifford gates).* In *Proceedings of the 21st International Conference on Quantum Physics and Logic, Buenos Aires, Argentina, July 15-19, 2024*, volume 406, pages 79–95. Open Publishing Association, 2024.

[37] Alex Townsend-Teague and collaborators. *Floquetifying the Colour Code.* arXiv: 2307.11136, 2023.

[38] Benjamin Rodatz, Boldizs'ar P'oor, and Aleks Kissinger. *Floquetifying stabiliser codes with distance-preserving rewrites.* arXiv: 2410.17240, 2024.

[39] Alex Townsend-Teague and Konstantinos Meichanetzidis. *Simplification Strategies for the Qutrit ZX-Calculus.* arXiv: 2103.06914, 2021.

[40] Tobias Fischbach, Pierre Talbot, and Pascal Bourvry. *A Review on Quantum Circuit Optimization using ZX-Calculus.* arXiv: 2509.20663, 2025.

[41] Tobias Fischbach, Pierre Talbot, and Pascal Bouvry. *Exhaustive Search for Quantum Circuit Optimization using ZX Calculus.* arXiv: 2503.10983, 2025.

[42] Benjamin Rodatz. *Fault Tolerance by Construction.* arXiv: 2506.17181, 2025.

[43] Craig Gidney. *Stim: a fast stabilizer circuit simulator.* Quantum, 5:497, 2021.

[44] Daniel Gottesman. *The Heisenberg representation of quantum computers.* arXiv: quant-ph/9807006, 1998.

[45] H. Bombin, D. Litinski, N. Nickerson, F. Pastawski, and S. Roberts. *Unifying flavors of fault tolerance with the ZX calculus.* arXiv: 2303.08829, 2023.

[46] Min-Hsiu Hsieh and François Le Gall. *NP-hardness of decoding quantum error-correction codes.* arXiv: 1009.1319, 2011.

[47] Quanlong Wang Kang Feng Ng. *A universal completion of the ZX-calculus*. arXiv: 1706.09877, 2017.

[48] F. Thomsen, M. S. Kesselring, S. D. Bartlett, and B. J. Brown. *Low-overhead quantum computing with the color code*. Physical Review Research, 6:043125, 2024.

[49] A. J. Landahl and C. Ryan-Anderson. *Quantum computing by color-code lattice surgery*. arXiv: 1407.5103, 2014.

[50] A. G. Fowler. *Two-dimensional color-code quantum computation*. Physical Review A, 83:042310, 2011.

[51] A. J. Landahl, J. T. Anderson, and P. R. Rice. *Fault-tolerant quantum computing with color codes*. arXiv: 1108.5738, 2011.

[52] L. S. Herzog, L. Berent, A. Kubica, and R. Wille. *Lattice surgery compilation beyond the surface code*. arXiv: 2504.10591, 2025.

[53] H. Bombin and M. A. Martin-Delgado. *Topological quantum color codes*. Physical Review Letters, 97(18), 2006.

[54] Craig Gidney and Cody Jones. *New circuits and an open source decoder for the color code*. Submitted on 14 Dec 2023, 2023.

[55] Joschka Roffe et al. *Decoding Across the Quantum LDPC Code Landscape*. arXiv: 2005.07016, 2020.

[56] Andrew M. Steane. *Active stabilization, quantum computation and quantum state synthesis*. Physical Review Letters, 78:2252, 1997.

[57] Yugo Takada and Keisuke Fujii. *Improving threshold for fault-tolerant color code quantum computing by flagged weight optimization*. Submitted on 21 Feb 2024 (v1), last revised 17 Sep 2024 (this version, v2), 2024.

[58] Coen Borghans. *ZX-Calculus and Quantum Stabilizer Theory*. Master's thesis, Radboud University, 2022.

[59] https://quantum.cloud.ibm.com/docs/en/guides/error-mitigation-and-suppression-techniques.